



به نام خدا
دانشگاه تهران
دانشکده مهندسی
برق و کامپیوتر



درس شبکه‌های عصبی و یادگیری عمیق
تمرین اول

پرسش ۱	نام و نام خانوادگی	سید مانی میرشعبانی
	شماره دانشجویی	۸۱۰۸۰۱۰۸۰
پرسش ۲	نام و نام خانوادگی	معین اکبری شهپر
	شماره دانشجویی	۸۱۰۸۰۱۰۵۵
	مهلت ارسال پاسخ	۱۴۰۳.۱۲.۲۷

فهرست

قوانین.....	۱
پرسش ۱ - تشخیص تقلب در کارت های اعتباری با استفاده از شبکه های عصبی چند لایه (MLP).....	۳
مقدمه.....	۳
پیش پردازش و بررسی دادگان.....	۳
طراحی و پیاده سازی یک شبکه MLP ساده.....	۸
تحلیل ماتریس آشفتگی و معیار های ارزیابی.....	۲۱
جست و جوی بهترین هایپرپارامترها شبکه یک لایه مخفی با Grid Search.....	۲۳
جمع بندی.....	۲۸
پرسش ۲ - طراحی شبکه عصبی چندلایه در مسئله رگرسیون مقاومت بتن.....	۳۱
مقدمه.....	۳۱
آماده سازی دادگان و تحلیل آماری.....	۳۱
بررسی همبستگی ویژگی ها با یکدیگر.....	۳۹
بررسی همبستگی ویژگی ها با مقاومت بتن.....	۴۱
پیاده سازی مدل شبکه عصبی چند لایه.....	۴۳
بررسی تغییرات تنظیمات مدل.....	۴۶
جمع بندی.....	۵۳
پرسش ۳. پیاده سازی Adaline برای دیتاست IRIS.....	۵۵
مقدمه.....	۵۵
آشنایی با Adaline.....	۵۵
تفاوت Adaline و MLP.....	۵۶
آماده سازی دادگان.....	۵۶
پیاده سازی و آموزش مدل Adaline.....	۵۸

۶۵	تحلیل نتایج.....
۶۷	پرسش ۴. آموزش اتوانکودر و طبقه بندی با دیتاست MNIST.....
۶۷	مقدمه
۶۷	دانلود و پیش پردازش داده ها
۶۸	طراحی و پیاده سازی مدل
۷۲	نتایج و تحلیل
۷۴	تحلیل و مقایسه

شکل‌ها

۴.....	شکل ۱
۵.....	شکل ۲
۶.....	شکل ۳
۷.....	شکل ۴
۷.....	شکل ۵
۹.....	شکل ۶
۱۰.....	شکل ۷
۱۰.....	شکل ۸
۱۱.....	شکل ۹
۱۲.....	شکل ۱۰
۱۳.....	شکل ۱۱
۱۵.....	شکل ۱۲
۱۶.....	شکل ۱۳
۱۶.....	شکل ۱۴
۱۷.....	شکل ۱۵
۳۱.....	شکل ۱۶
۳۲.....	شکل ۱۷
۳۲.....	شکل ۱۸
۳۳.....	شکل ۱۹
۳۴.....	شکل ۲۰
۳۴.....	شکل ۲۱
۳۵.....	شکل ۲۲
۳۵.....	شکل ۲۳
۳۶.....	شکل ۲۴
۳۶.....	شکل ۲۵
۳۷.....	شکل ۲۶
۳۸.....	شکل ۲۷

شکل ۲۸.....	۳۹.....
شکل ۲۹.....	۳۹.....
شکل ۳۰.....	۴۱.....
شکل ۳۱.....	۴۲.....
شکل ۳۲.....	۴۳.....
شکل ۳۳.....	۴۴.....
شکل ۳۴.....	۴۵.....
شکل ۳۵.....	۴۵.....
شکل ۳۶.....	۴۶.....
شکل ۳۷.....	۴۶.....
شکل ۳۸.....	۴۷.....
شکل ۳۹.....	۴۸.....
شکل ۴۰.....	۴۸.....
شکل ۴۱.....	۴۹.....
شکل ۴۲.....	۴۹.....
شکل ۴۳.....	۵۰.....
شکل ۴۴.....	۵۱.....
شکل ۴۵.....	۵۱.....
شکل ۴۶.....	۵۳.....
شکل ۴۷.....	۵۵.....
شکل ۴۸.....	۵۷.....
شکل ۴۹.....	۵۷.....
شکل ۵۰.....	۵۸.....
شکل ۵۱.....	۶۰.....
شکل ۵۲.....	۶۱.....
شکل ۵۳.....	۶۲.....
شکل ۵۴.....	۶۳.....
شکل ۵۵.....	۶۴.....
شکل ۵۶.....	۶۵.....

۶۷.....	شکل ۵۷.....
۶۷.....	شکل ۵۸.....
۶۸.....	شکل ۵۹.....
۶۹.....	شکل ۶۰.....
۷۰.....	شکل ۶۱.....
۷۱.....	شکل ۶۲.....
۷۱.....	شکل ۶۳.....
۷۳.....	شکل ۶۴.....

جدول‌ها

جدول ۱ ۱۲

جدول ۲ ۲۴

قبل از پاسخ دادن به پرسش‌ها، موارد زیر را با دقت مطالعه نمایید:

- از پاسخ‌های خود یک گزارش در قالبی که در صفحه‌ی درس در سامانه‌ی Elearn با نام **REPORTS_TEMPLATE.docx** قرار داده شده تهیه نمایید.
- پیشنهاد می‌شود تمرین‌ها را در قالب گروه‌های دو نفره انجام دهید. (بیش از دو نفر مجاز نیست و تحویل تک نفره نیز نمره‌ی اضافی ندارد) توجه نمایید الزامی در یکسان ماندن اعضای گروه تا انتهای ترم وجود ندارد. (یعنی، می‌توانید تمرین اول را با شخص A و تمرین دوم را با شخص B و ... انجام دهید)
- **کیفیت گزارش شما در فرآیند تصحیح از اهمیت ویژه‌ای برخوردار است؛** بنابراین، لطفاً تمامی نکات و فرض‌هایی را که در پیاده‌سازی‌ها و محاسبات خود در نظر می‌گیرید در گزارش ذکر کنید.
- در گزارش خود مطابق با آنچه در قالب نمونه قرار داده شده، برای شکل‌ها زیرنویس و برای جدول‌ها بالانویس در نظر بگیرید.
- الزامی به ارائه توضیح جزئیات کد در گزارش نیست، اما باید نتایج بدست آمده از آن را گزارش و تحلیل کنید.
- **تحلیل نتایج الزامی می‌باشد، حتی اگر در صورت پرسش اشاره‌ای به آن نشده باشد.**
- **دستیاران آموزشی ملزم به اجرا کردن کدهای شما نیستند؛** بنابراین، هرگونه نتیجه و یا تحلیلی که در صورت پرسش از شما خواسته شده را به طور واضح و کامل در گزارش بیاورید. در صورت عدم رعایت این مورد، بدیهی است که از نمره تمرین کسر می‌شود.
- **کدها حتماً باید در قالب نوت‌بوک با پسوند .ipynb تهیه شوند، در پایان کار، تمامی کد اجرا شود و خروجی هر سلول حتماً در این فایل ارسالی شما ذخیره شده باشد.** بنابراین برای مثال اگر خروجی سلولی یک نمودار است که در گزارش آورده‌اید، این نمودار باید هم در گزارش هم در نوت‌بوک کدها وجود داشته باشد.
- **در صورت مشاهده‌ی تقلب امتیاز تمامی افراد شرکت‌کننده در آن، 100- لحاظ می‌شود.**
- تنها زبان برنامه نویسی مجاز **Python** است.
- **استفاده از کدهای آماده برای تمرین‌ها به هیچ وجه مجاز نیست. در صورتی که دو گروه از یک منبع مشترک استفاده کنند و کدهای مشابه تحویل دهند، تقلب محسوب می‌شود.**
- نحوه محاسبه تاخیر به این شکل است: پس از پایان رسیدن مهلت ارسال گزارش، حداکثر تا یک هفته امکان ارسال با تاخیر وجود دارد، پس از این یک هفته نمره آن تکلیف برای شما صفر خواهد شد.

○ سه روز اول: بدون جریمه

○ روز چهارم: ۵ درصد

○ روز پنجم: ۱۰ درصد

○ روز ششم: ۱۵ درصد

○ روز هفتم: ۲۰ درصد

● حداکثر نمره‌ای که برای هر سوال می‌توان اخذ کرد ۱۰۰ بوده و اگر مجموع بارم یک سوال بیشتر

از ۱۰۰ باشد، در صورت اخذ نمره بیشتر از ۱۰۰، اعمال نخواهد شد.

○ برای مثال: اگر نمره اخذ شده از سوال ۱ برابر ۱۰۵ و نمره سوال ۲ برابر ۹۵ باشد، نمره نهایی

تمرین ۹۷.۵ خواهد بود و نه ۱۰۰.

● لطفا گزارش، کدها و سایر ضمایم را به در یک پوشه با نام زیر قرار داده و آن را فشرده سازید،

سپس در سامانه‌ی Elearn بارگذاری نمایید:

HW[Number]_[Lastname]_[StudentNumber]_[Lastname]_[StudentNumber].zip

(مثال: HW1_Ahmadi_810199101_Bagheri_810199102.zip)

● برای گروه‌های دو نفره، بارگذاری تمرین از جانب یکی از اعضا کافی است ولی پیشنهاد می‌شود هر

دو نفر بارگذاری نمایند.

پرسش ۱ - تشخیص تقلب در کارت های اعتباری با استفاده از شبکه های عصبی چند لایه (MLP)

مقدمه

در این بخش توضیحاتی درباره سوال داده شده بود و چیزی برای پاسخگویی وجود نداشت.

پیش پردازش و بررسی دادگان

دانلود دیتاست

با توجه به اینکه مدل را به علت عدم دسترسی به اینترنت پایدار به صورت local اجرا کردیم، داده ها از kaggle دانلود شده و در پوشه مربوط به کد ها قرار گرفتند.

نمایش خلاصه از داده ها

داده ها به صورت دیتافریم در کد لود شده و به کمک دستوراتی چون head() و describe() و ... اطلاعاتی از آنها نمایش داده شد که در شکل ۱.۱ میتوان آن را مشاهده کرد
با توجه به اطلاعات بدست آمده میتوان دید که ورودی چند متغیر دارد (۲۹) و توزیع هر کدام به چه صورت است، همچنین میتوان دید که این دیتاست دو کلاس اصلی دارد و همه دیتا موجود است (NaN نداریم)

- نمودار میله ای به کمک قطعه کد زیر رسم شده، همانطور که دیده میشود اکثریت دیتا از کلاس (Non Fraud) هستند (۹۹.۸٪)

1-2-3 Highlighting class imbalances using bar charts:



شکل ۱

```
import pandas as pd

data_path = "../creditcard.csv"

credit_card_data = pd.read_csv(data_path)

print("First 5 rows: ")
credit_card_data.head()
```

First 5 rows:

V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

```
print("Last 5 rows: ")
credit_card_data.tail()
```

Last 5 rows:

V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
7.305334	1.914428	...	0.213454	0.111864	1.014480	-0.509348	1.436807	0.250034	0.943651	0.823731	0.77	0
0.294869	0.584800	...	0.214205	0.924384	0.012463	-1.016226	-0.606624	-0.395255	0.068472	-0.053527	24.79	0
0.708417	0.432454	...	0.232045	0.578229	-0.037501	0.640134	0.265745	-0.087371	0.004455	-0.026561	67.88	0
0.679145	0.392087	...	0.265245	0.800049	-0.163298	0.123205	-0.569159	0.546668	0.108821	0.104533	10.00	0
-0.414650	0.486180	...	0.261057	0.643078	0.376777	0.008797	-0.473649	-0.818267	-0.002415	0.013649	217.00	0

```
print("Dataset info: ")
credit_card_data.info()
```

Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
Column Non-Null Count Dtype

0 Time 284807 non-null float64

```
print("Statistical Summary: ")
credit_card_data.describe()
```

Statistical Summary:

	Time	V1	V2	V3	V4	V5	V6
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.604066e-16	1.487313e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01

```
print("Missing values (NaNs) :")
credit_card_data.isnull().sum()
```

Missing values (NaNs) :

Time	0
V1	0
V2	0
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0

شكل ٢

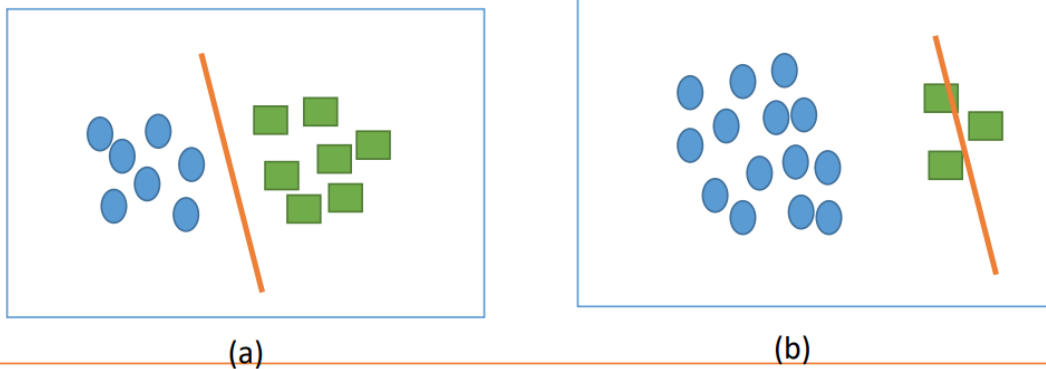
چالش عدم تعادل کلاس ها

همانطور که در کلاس مطرح شد، در شبکه هایی مثل Adaline عدم توازن کلاس ها میتواند منجر به اشتباه جواب دادن در شبکه شود (شکل ۳) علاوه بر این تعداد بالای یک نوع کلاس میتواند به هنگام استفاده از معیار هایی مثل MSE شبکه را به سمتی سوق دهد که اکثریت دیتا در آن است و این امر منجر به کاهش قابلیت generalization شبکه شود!

علاوه بر اینها، عدم توازن میتواند معیار های ارزیابی ما از شبکه را هم خراب کند چرا که مثلاً در همین مثال خاص، اگر شبکه به ازای همه ورودی ها عدد ۰ را برگرداند با دقت ۹۹٪ درست جواب داده است! در حالی که اصلاً به هدف اصلی خود یعنی تشخیص Fraud نزدیک نشده

نداشتن معیار های مناسب برای تشخیص درستی شبکه علاوه بر این ارزیابی ما از شبکه را سخت میکند منجر به سخت شدن train شبکه هم میشود چراکه نمیتوان ارزیابی درستی از treshhold های شبکه داشت.

به همین جهت باید برای حفظ تعادل کلاس ها در شبکه در حالتی که اختلاف زیادی وجود دارند چاره ای اندیشید.



شکل ۳

نرمال سازی ویژگی ها

برای عملکرد بهتر شبکه و جلوگیری از Bias به علت بزرگی یا کوچکی اعداد، دیتاست به کمک StandardScaler استاندارد سازی شده و نتیجه نشان داده شد.

1-2-5 Normalizing the data:

For better performance the data has been normalized using StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
features = credit_card_data.drop(columns=["Class"])
normalized_features = scaler.fit_transform(features)

normalized_credit_card_data = pd.DataFrame(
    normalized_features,
    columns=features.columns
)
normalized_credit_card_data["Class"] = credit_card_data["Class"]

normalized_credit_card_data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21
0	-1.996583	-0.694242	-0.044075	1.672773	0.973366	-0.245117	0.347068	0.193679	0.082637	0.331128	...	-0.024923
1	-1.996583	0.608496	0.161176	0.109797	0.316523	0.043483	-0.061820	-0.063700	0.071253	-0.232494	...	-0.307377

شکل ۴

تقسیم دیتاست به test و training

با استفاده از کتابخانه scikit-learn دیتا با حفظ توازن در کلاس ها (طبق صورت سوال) تقسیم شده است.

1-2-6 Splitting the dataset

The data set has been split to 70% training and 30% testing data.

```
from sklearn.model_selection import train_test_split

x = standardized_credit_card_data.drop(columns=['Class'])
y = standardized_credit_card_data['Class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=24, stratify=y
)
```

شکل ۵

طراحی و پیاده سازی یک شبکه MLP ساده

تعریف شبکه با مشخصات داده شده

شبکه با مشخصات داده شده به کمک کد زیر تعریف شد علاوه بر این برای عدم تکرار کد، قابلیت اضافه کردن drop out و regularization هم به تابعی که مدل را تولید میکند اضافه شده است (شکل ۱.۶) در تعریف این شبکه از keras.Sequential استفاده شد که به ما اجازه میدهد لایه ها را به ترتیب اضافه کنیم.

در مورد بخش های dropout و regularization هم با توجه به اینکه در کلاس صحبتی از آنها نشده بود، خلاصه ای از نتایجی که با جست و جو بدست آوردم رو در ادامه آوردم:

Dropout: نورون های شبکه را در حین train به صورت تصادفی خاموش میکند، این امر به شبکه کمک میکند حساسیت کمتری به نویز داشته باشد و دنبال ویژگی های اصلی بگردد و جلوی overfit شدن را بگیرد.

Regularization: با اضافه کردن عباراتی چون جمع مربع وزن ها به تابع loss باعث میشود که وزن های شبکه خیلی زیاد نشوند (چرا که زیاد شدن وزن ها حساسیت شبکه به ورودی را بالا برده و منجر میشود شبکه overfit شود!)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

input_columns = x_train.shape[1]

def create_mlp(dropout_rate=None, l2_regularization=None):
    model = keras.Sequential()

    if l2_regularization:
        model.add(
            layers.Dense(
                64,
                activation='relu',
                kernel_regularizer=regularizers.l2(l2_regularization),
                input_shape=(input_columns,)
            )
        )
    else:
        model.add(
            layers.Dense(
                64,
                activation='relu',
                kernel_regularizer=None,
                input_shape=(input_columns,)
            )
        )

    if dropout_rate:
        model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1, activation='sigmoid'))

    return model
```

شكل ٦

چهار مدل مختلف (با و بدون dropout و regularization) با استفاده از قطعه کد زیر (شکل ۱.۷) و استفاده از Binary Cross-Entropy به عنوان تابع هزینه در ۴۰ epoch و با batch size به اندازه ۴۰ train شدند.

1-3-2 Training the model

For training the model, Binary Cross-Entropy has been chosen as the loss function and Adam optimizer has been utilized.

- Binary Cross-Entropy: $\text{Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
- Adam optimizer: Adaptive Moment Estimation is a popular optimizer for MLPs, it adapts the learning rate for each parameter dynamically

The model has been trained for 40 epoch with the batch size of 32

4 models have been trained in total, to see the effects of dropout and regularizer

```

models = {"plain_model": create_model(
    dropout_rate=None,
    l2_regularization=None
),

"dropout_model": create_model(
    dropout_rate=0.3,
    l2_regularization=None
),

"regularizer_model": create_model(
    dropout_rate=None,
    l2_regularization=0.0001
),

"dropout_regularizer_model": create_model(
    dropout_rate=0.3,
    l2_regularization=0.0001
)}

```

```

model_data = dict()

for model_name, model in models.items():
    model.compile(
        loss="binary_crossentropy",
        optimizer="adam",
        metrics=["accuracy"]
    )

    model_data[model_name] = model.fit(
        x_train,
        y_train,
        epochs=40,
        batch_size=32,
        validation_data=(x_test, y_test)
    )

```

شکل ۷

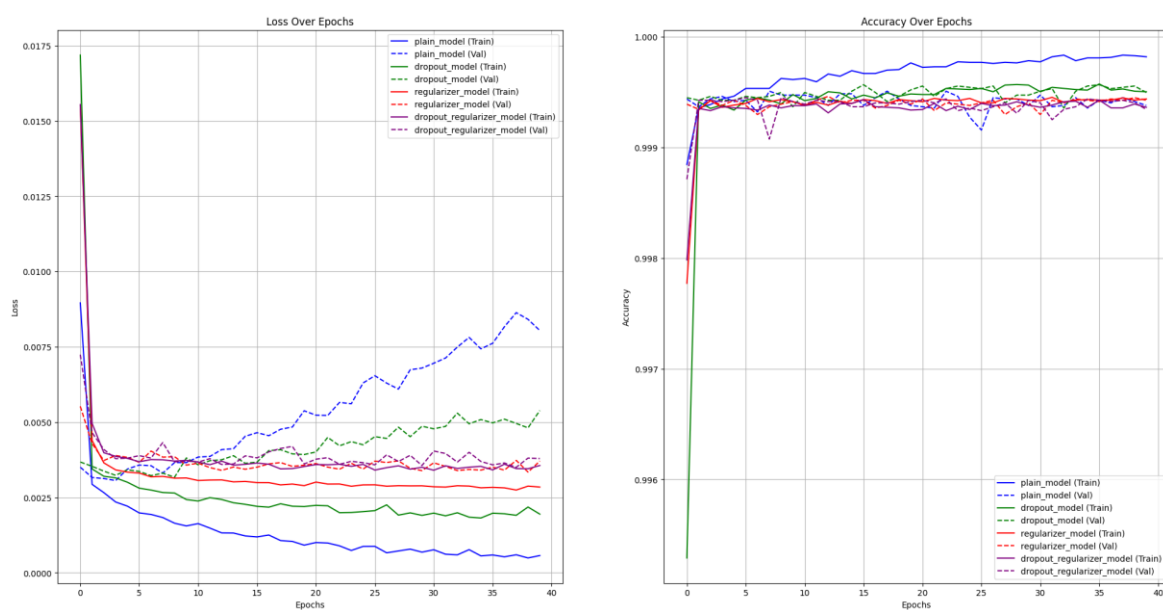
```

1 import matplotlib.pyplot as plt
2
3 colors = {
4     "plain_model": "blue",
5     "dropout_model": "green",
6     "regularizer_model": "red",
7     "dropout_regularizer_model": "purple"
8 }
9
10 plt.figure(figsize=(24, 12))
11 plt.subplot(1, 2, 1)
12 for model_name, records in model_data.items():
13     plt.plot(records.history["loss"], linestyle='-', label=f"{model_name} (Train)", color=colors[model_name])
14     plt.plot(records.history["val_loss"], linestyle="--", label=f"{model_name} (Val)", color=colors[model_name])
15
16 plt.xlabel("Epochs")
17 plt.ylabel("Loss")
18 plt.title("Loss Over Epochs")
19 plt.legend()
20 plt.grid()
21
22 plt.subplot(1, 2, 2)
23 for model_name, records in model_data.items():
24     plt.plot(records.history["accuracy"], linestyle='-', label=f"{model_name} (Train)", color=colors[model_name])
25     plt.plot(records.history["val_accuracy"], linestyle="--", label=f"{model_name} (Val)", color=colors[model_name])
26
27 plt.xlabel("Epochs")
28 plt.ylabel("Accuracy")
29 plt.title("Accuracy Over Epochs")
30 plt.legend()
31 plt.grid()
32
33 plt.show()
34

```

شکل ۸

یکی از روش هایی که میتوان به کمک آن قابلیت تعمیم شبکه را بررسی کرد و متوجه رخ دادن overfitting شد، بررسی نمودار های loss و accuracy است. اگر برای دیتای train در حال پایین رفتن باشد ولی برای validation بالا برود، میتواند نشان دهنده این باشد که شبکه اطلاعات را حفظ کرده و overfit شده است! (به طور مشابه هم میتوان برای نمودار accuracy بررسی کرد) به کمک این نمودار زیر را تحلیل میکنیم:



شکل ۹

با توجه به نمودار میتوان دید که شبکه ای که از هیچ کدام از dropout و regularization استفاده نکرده بود (آبی رنگ)، به سرعت overfit میشود.

مدل سبز رنگ هم که فقط از dropout استفاده کرده بود بعد از مدتی overfit میشود که میتواند نشان دهنده این باشد که استفاده از dropout به تنهایی برای جلوگیری از overfit کافی نیست، چراکه همچنان ممکن است وزن های شبکه بزرگ شده و شبکه به نویز حساس شود. به طور کلی مدل هایی که از regularization استفاده کرده بودند (قرمز و بنفش) نتیجه بهتری داشتند، هرچند accuracy برای دیتاست هایی که توازن پایینی دارند معیار خوبی نیست، در بخش بعدی سایر معیار ها برای مقایسه شبکه ها استفاده می شوند. برای ارزیابی مدل سه مرحله داریم که در ادامه بررسی می شوند.

ماتریس آشفته‌گی:

این ماتریس نشان می‌دهد که مدل ما چه تعدادی از هر کلاس را درست و چه تعدادی را غلط تشخیص داده است و به صورت جدول زیر محاسبه می‌شود. این ماتریس به کمک میکند درک بهتری از مدل پیدا کنیم (در مقابل اینکه فقط از Accuracy استفاده کنیم) چرا که به ما نشان می‌دهد خطاها در تشخیص کدام کلاس‌ها بوده، لذا برای دیتاست‌های نامتوازن معیار مناسب‌تری است.

جدول ۱

Actual \ Predicted	Predicted No	Predicted Yes
Actually No	True Negative(TN)	False Positive(FP)
Actually Yes	False Negative(FN)	True Positive(TP)

در این جدول برای هر یک از عبارات داریم:

TN: تعداد مواردی که جواب منفی بوده و مدل آنها را به درستی منفی پیش‌بینی کرده

FP: تعداد مواردی که جواب منفی بوده و مدل آنها را به اشتباه مثبت پیش‌بینی کرده

FN: تعداد مواردی که جواب مثبت بوده و مدل آنها را به اشتباه منفی پیش‌بینی کرده

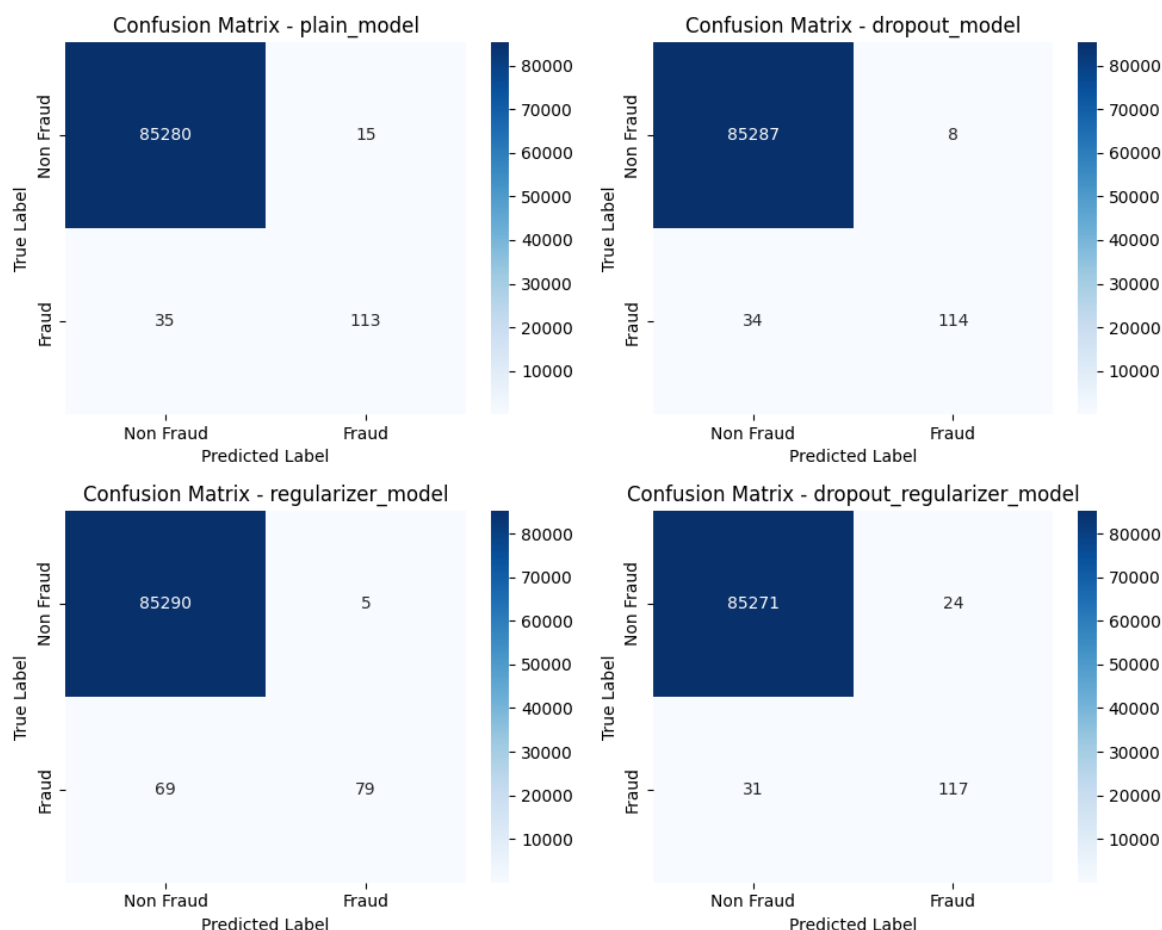
TP: تعداد مواردی که جواب مثبت بوده و مدل آنها را به درستی مثبت پیش‌بینی کرده

ماتریس آشفته‌گی برای ۴ مدلی بالاتر تعریف شدند به کمک قطعه کد شکل ۱۰.۱۰ محاسبه شده و نتایج

آن را میتوان در شکل ۱۰.۱۱ مشاهده کرد.

```
1 import sklearn.metrics
2
3 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
4 axes = axes.flatten()
5 for i, (model_name, model) in enumerate(models.items()):
6     y_hat = (model.predict(x_test) > 0.5).astype("int64")
7     confusion_matrix = sklearn.metrics.confusion_matrix(y_test, y_hat)
8
9     print(model_name)
10    print(confusion_matrix)
11    sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=["Non Fraud", "Fraud"], yticklabels=["Non Fraud", "Fraud"], ax=axes[i])
12    axes[i].set_xlabel("Predicted Label")
13    axes[i].set_ylabel("True Label")
14    axes[i].set_title(f"Confusion Matrix - {model_name}")
15
16 plt.tight_layout()
17 plt.show()
```

شکل ۱۰



شکل ۱۱

همانطور که از شکل ۱.۱۱ مشخص است تعداد TN ها بسیار بالا است که با توجه به نا متوازن بودن دیتاست این انتظار میرفت.

بعد از چندن بار ترین کردن مدل ها، تفاوت چشم گیری میان مدل ها نبود ولی به طور متوسط مدل dropout بهترین نتیجه کلی را میگرفت، مدلی که فقط از ruglarizer استفاده میکرد واریانس زیادی داشت و نتایج بسیار متفاوتی میداد و مدلی که از هر دوی dropout و regualirzation استفاده میکرد نسبتا ثابت ثابت عمل میکرد و نتایج قابل قبولی میگرفت

بررسی Accuracy، Precision و Recall و F1-Score

Accuracy: درستی کلی مدل را نشان میدهد به صورت نسبت تعداد پیشبینی های درست مدل به کلی ورودی ها گفته میشود.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: نشان میدهد از بین تمام مواردی که خروجی شبکه ۱ بوده، چه کسری از آنها واقعا ۱

بوده اند.

$$Precision = \frac{TP}{TP + FP}$$

Recall: نشان میدهد از بین تمام موارد ۱ در ورودی، چه کسری از آنها به درستی ۱ پیشبینی شده

اند.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score: از آنجایی که حداکثر کردن یکی از Recall یا Precision منجر به کمتر شدن دیگری

میشود (مثلا اگر مدل به همه وردی ها خروجی ۱ بدهد، Recall حداکثر میشود ولی Precision به شدت

افت میکند) برای همین معیار F1-Score به صورت میانگین هارمونیک این دو متغیر تعریف می شود.

هر چقدر این معیار بیشتر باشد یعنی هر دوی Recall و Precision بالا تر بوده اند.

$$F1 - Score = 2 \frac{Precision * Recall}{Precision + Recall}$$

این معیار ها به کمک قطعه کد شکل ۱۲ محاسبه شدند و نتایج آنها را میتوان در این شکل دید. با توجه به این نتایج مدلی که از dropout استفاده می کرد بهترین نتیجه را به طور کلی داشت ولی به غیر از مدل regularization بقیه مدل ها نتایج نسبتاً نزدیکی به هم داشتند.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

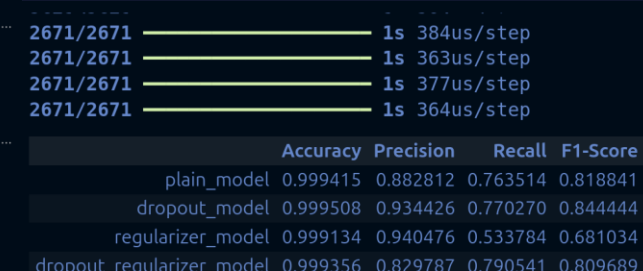
metrics = {}

for model_name, model in models.items():
    y_hat = (model.predict(x_test) > 0.5).astype("int64")

    accuracy = accuracy_score(y_test, y_hat)
    precision = precision_score(y_test, y_hat)
    recall = recall_score(y_test, y_hat)
    f1 = f1_score(y_test, y_hat)

    metrics[model_name] = {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    }

pd.DataFrame(metrics).T
```



	Accuracy	Precision	Recall	F1-Score
plain_model	0.999415	0.882812	0.763514	0.818841
dropout_model	0.999508	0.934426	0.770270	0.844444
regularizer_model	0.999134	0.940476	0.533784	0.681034
dropout_regularizer_model	0.999356	0.829787	0.790541	0.809689

شکل ۱۲

منحنی ROC و مقدار AUC

قبل از بررسی این مقادیر ابتدا توضیح مختصری درباره اشان می‌دهیم:

منحنی ROC: این منحنی رابطه (Trade off) میان True Positive Rate و False Positive Rate را با توجه به تغییر threshold نشان می‌دهد، به کمک این نمودار میتوان تصویری از درستی مدل داشت. هرچقدر نمودار بیشتر به سمت بالا چپ برود یعنی مدل بهتری داریم.

مقدار AUC: سطح زیر نمودار ROC را نشان می‌دهد که معیاری برای بررسی اینکه مدل چقدر خوب میتواند بین کلاس ها تفاوت قائل شود است. هرچقدر این مقدار به ۱ نزدیک تر باشد بهتر است.

توضیحات بیشتر درباره این دو مقدار را میتوان در شکل ۱۳ دید.

First lets give a brief explanation:

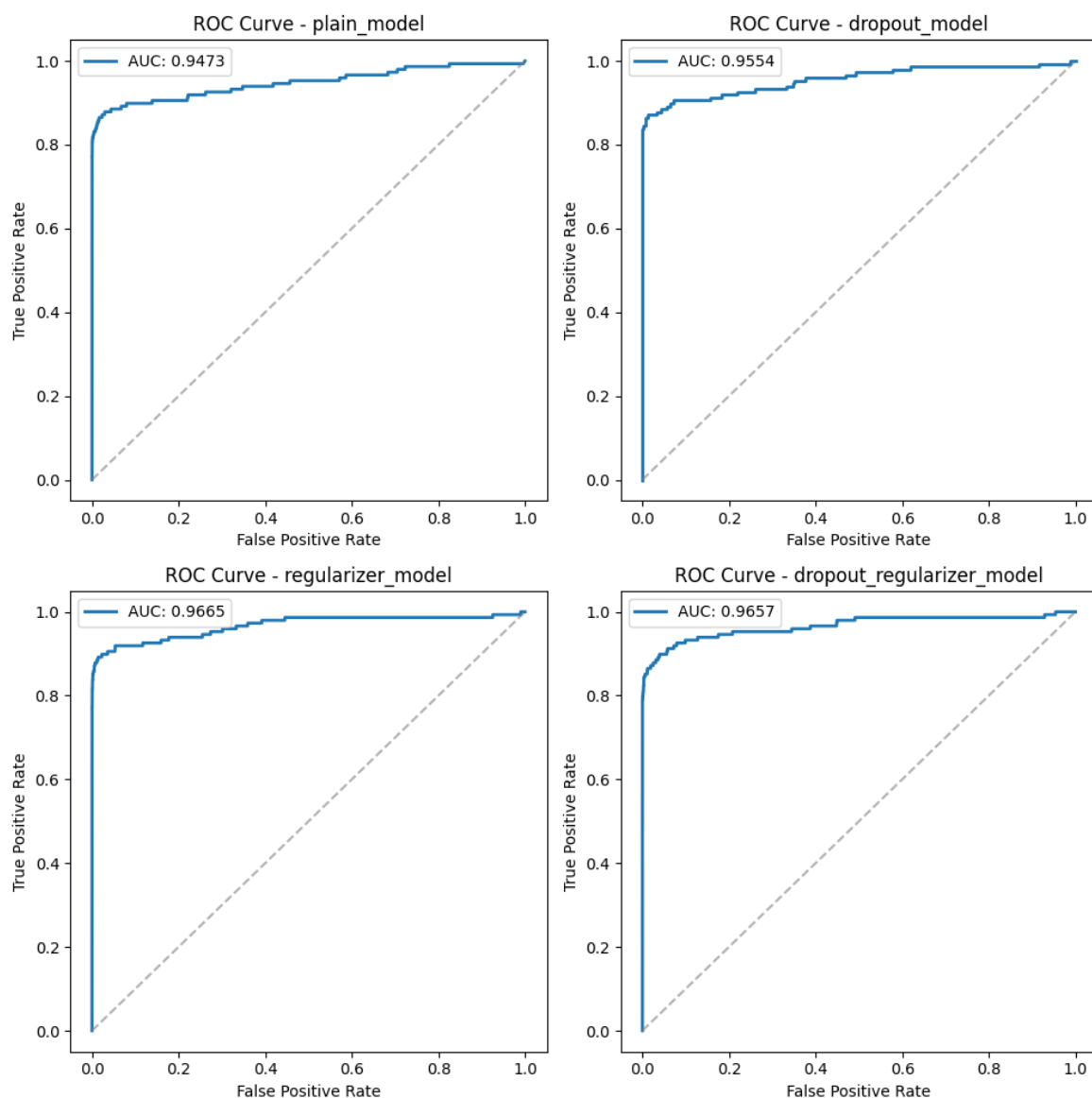
- **ROC Curve:** Receiver Operating Characteristic Curve (ROC Curve) is a plot that shows the trade of between **True Positive Rate** and **False Positive Rate** for different thresholds. It is plotted by changing the decision threshold of the model.
- **True Positive Rate:** The number of TPs divided by the total number of Positives in the input
 - $TPR = \frac{TP}{TP+FN}$
- **False Positive Rate:** The number of FPs divided by the total number of Negatives in the input
 - $FPR = \frac{FP}{FP+TN}$
- **AUC Score:** Area Under Curve score is a performance metric used in classification models, especially for imbalanced datasets. It measures how well the model distinguishes between positive and negative classes. It is the total area under the ROC curve, representing the probability that a randomly chosen fraud case is ranked higher than a randomly chosen non-fraud case.
 - AUC = 1.0 (Perfect Model) → Always ranks fraud cases higher than non-fraud cases.
 - AUC = 0.5 (Random Guessing) → Model has no discrimination ability.
 - AUC < 0.5 (Worse than Random) → Model is misclassifying more than a random classifier.

شکل ۱۳

این مقادیر برای مدل ها به کمک قطعه کد زیر تولید شدند (شکل ۱۴) و نتایج آن در شکل ۱۵ قابل مشاهده است، همانطور که دیده میشود عملکرد مدل ها در این معیار بسیار به هم نزدیک است.

```
1 fig, axes = plt.subplots(2, 2, figsize=(10, 10))
2 axes = axes.flatten()
3
4 for i, (model_name, model) in enumerate(models.items()):
5     y_prediction_probability = model.predict(x_test)[: , 0]
6     fpr, tpr, _ = sklearn.metrics.roc_curve(
7         y_test,
8         y_prediction_probability
9     )
10
11     auc_score = sklearn.metrics.auc(fpr, tpr)
12
13     axes[i].plot(fpr, tpr, label=f"AUC: {auc_score:.4f}", linewidth=2)
14     axes[i].plot([0, 1], [0, 1], linestyle="--", color="gray", alpha=0.6)
15     axes[i].set_title(f"ROC Curve - {model_name}")
16     axes[i].set_xlabel("False Positive Rate")
17     axes[i].set_ylabel("True Positive Rate")
18     axes[i].legend()
19
20 plt.tight_layout()
21 plt.show()
```

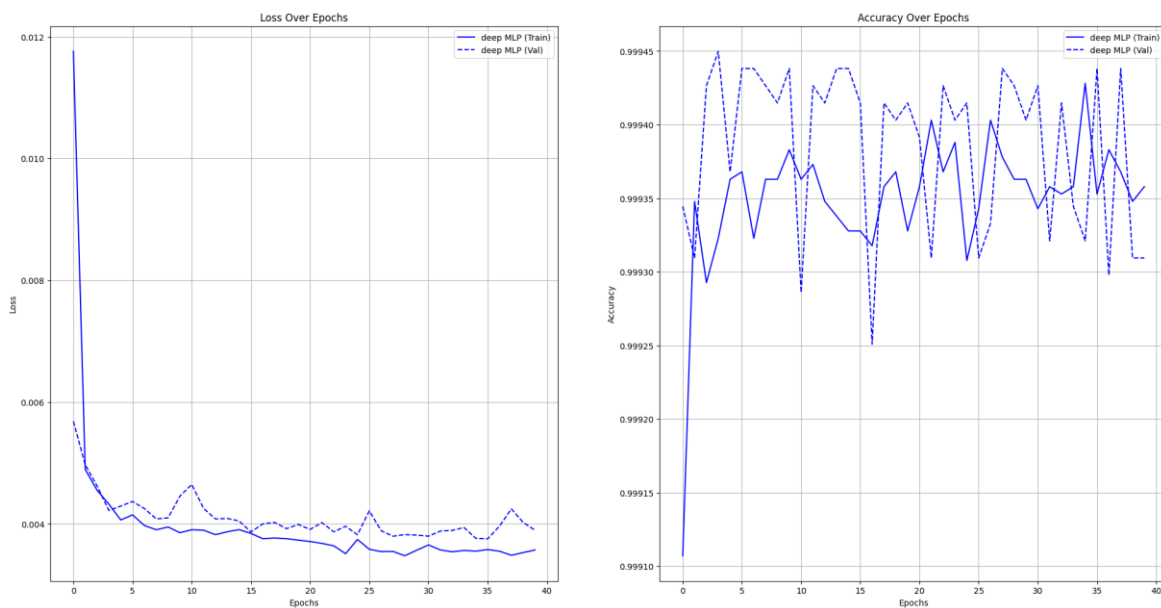
شکل ۱۴



شکل ۱۵

طراحی یک شبکه عصبی MLP عمیق تر

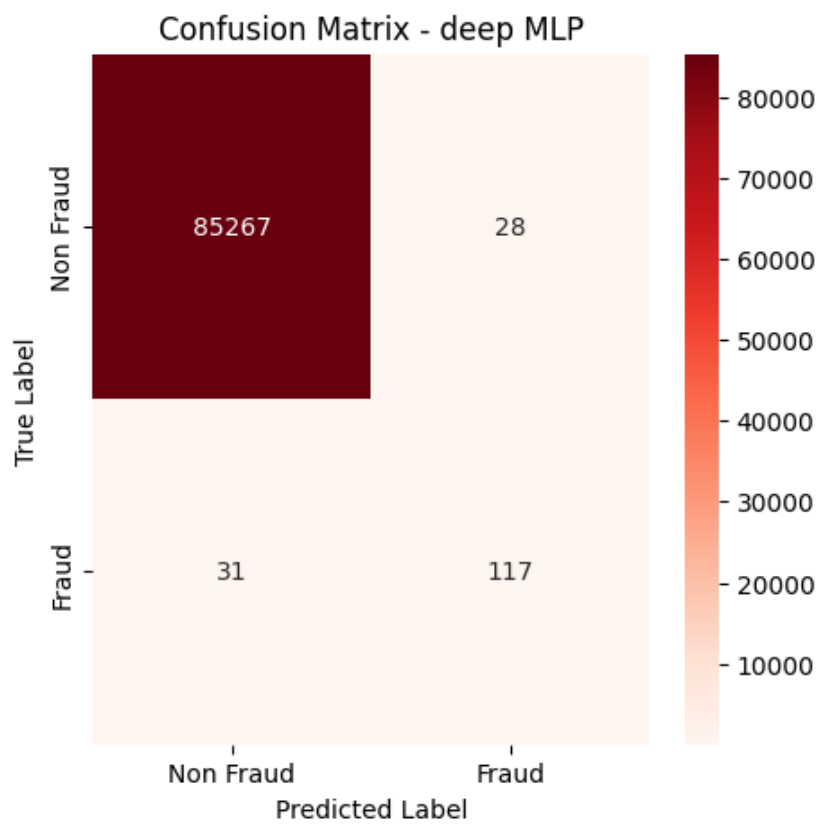
یک شبکه MLP با دولایه مخفی با توجه به توضیحات سوال (۱۲۸ نورون و ۶۴ نورون و فعالساز relu) طراحی و با استفاده از تابع زیان binary cross entropy و بهینه ساز adam آموزش داده شد. با توجه به شکل، مدل بدون بیش برازش و با دقت نسبتاً خوبی آموزش دید اما همانطور که از نمودار Accuracy مشخص است استفاده از این معیار برای داده های نامتوازن مناسب نیست.



شکل ۱۶

ارزیابی

مشابه با مدل قبلی معیار های ارزیابی مورد بررسی قرار می گیرند.



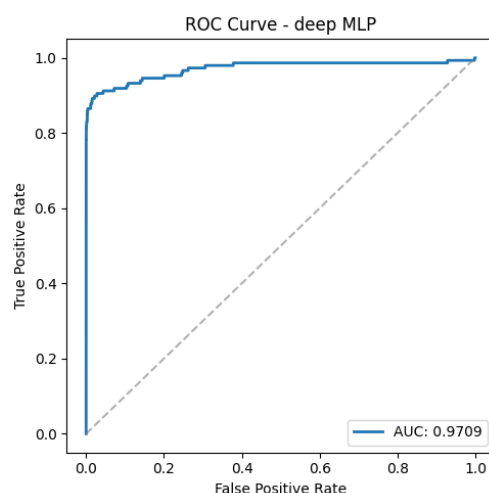
شکل ۱۷

همانطور که ماتریس آشفتگی نمایش می دهد، شبکه عمیق تر پیشرفتی نسبت به مدل قبل نداشته و حتی False Negative های بیشتری نیز دارد. این امر در شکل بعدی وضوح قابل مشاهده است که F1-Score مدل جدید و سایر معیار ها از مدل های قبلی بدتر است.

	Accuracy	Precision	Recall	F1-Score
plain_model	0.999391	0.863636	0.770270	0.814286
dropout_model	0.999438	0.931034	0.729730	0.818182
regularizer_model	0.999368	0.835714	0.790541	0.812500
dropout_regularizer_model	0.999415	0.871212	0.777027	0.821429
deep_model	0.999309	0.806897	0.790541	0.798635

شکل ۱۸

با توجه به نمودار ROC که به سمت گوشه بالا-چپ همگرا می باشد، مدل قدرت تشخیص خوبی دارد. این امر از سطح زیر نمودار (AUC) نیز قابل تایید است.



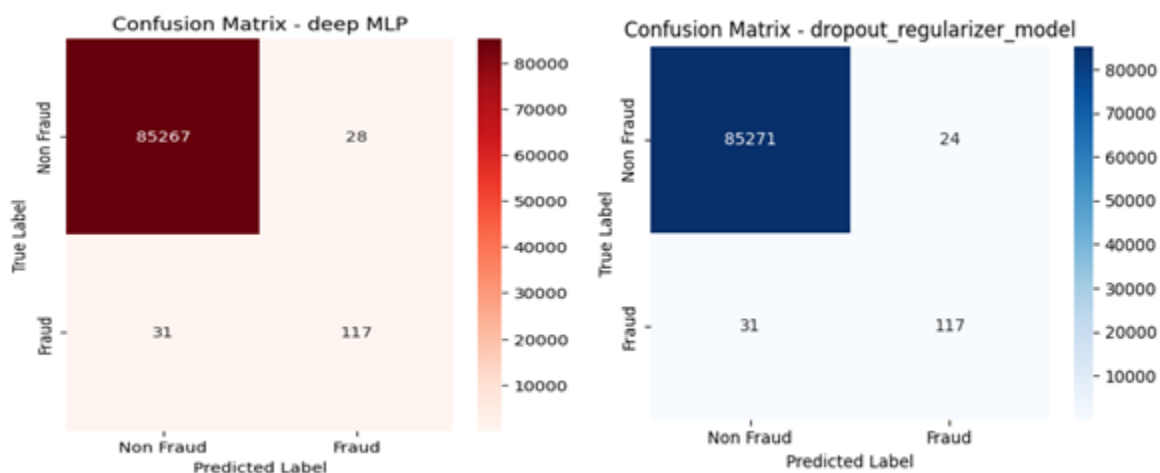
شکل ۱۹

مقایسه با مدل های قبلی

با توجه به معیار های قبلی، بهترین مدل آموزش داده شده، مدلی است که از dropout و regularization استفاده می کند؛ به همین جهت مدل عمیق را با این مدل مقایسه می کنیم.

- ماتریس آشفتگی

به طور کلی، مدل عمیق تر نسبت به مدل بهینه قبلی بهبود چندانی نداشته است، بلکه تعداد FP ها در شبکه عمیق، نسبت به مدل بهینه قبلی افزایش نیز داشته است.



شکل ۲۰

• F1-Score و Accuracy, Recall, Precision

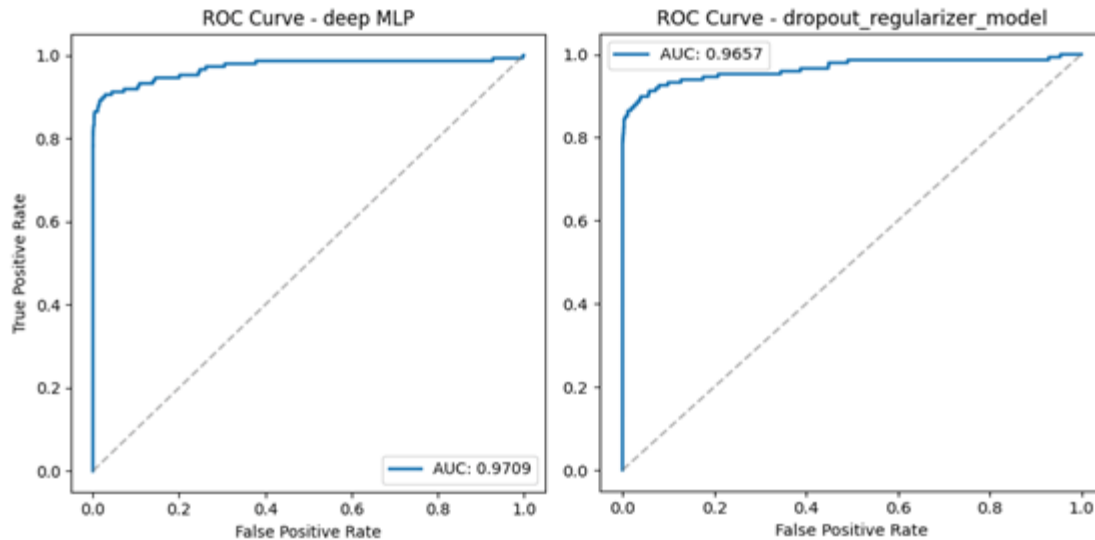
عدم بهبود عملکرد مدل عمیق، در معیار های عددی نیز مشهود است. این مدل کمترین F1-Score را در مقایسه با مدل های قبلی داراست.

	Accuracy	Precision	Recall	F1-Score
plain_model	0.999391	0.863636	0.770270	0.814286
dropout_model	0.999438	0.931034	0.729730	0.818182
regularizer_model	0.999368	0.835714	0.790541	0.812500
dropout_regularizer_model	0.999415	0.871212	0.777027	0.821429
deep_model	0.999309	0.806897	0.790541	0.798635

شکل ۲۱

• ROC و AUC

با وجود اینکه AUC مدل عمیق نسبت به مدل dropout_regularizer افزایش داشته است، اما این اختلاف ناچیز است و تاثیر چشمگیری در عملکرد مدل ندارد.



شکل ۲۲

تحلیل ماتریس آشفستگی و معیار های ارزیابی

دقت (accuracy):

تعداد نمونه‌هایی که مدل به درستی طبقه‌بندی کرده است نسبت به کل نمونه‌ها را با accuracy نشان می‌دهیم. در داده‌های نامتوازن معیار مناسبی نیست، زیرا می‌تواند مقدار بالایی نشان دهد حتی اگر مدل تنها کلاس غالب (در این مسئله، non-fraud) را برای تمامی نمونه‌ها پیش‌بینی کند.

Precision:

نشان می‌دهد از میان نمونه‌هایی که مدل مثبت پیش‌بینی کرده، چه کسری واقعاً مثبت هستند. در تشخیص تقلب مهم است، زیرا بالا بودن Precision به این معناست که تعداد خطای مثبت کاذب (False Positive) کم است و افرادی که تقلبی انجام نداده‌اند به اشتباه متقلب شناسایی نمی‌شوند. اگر Precision پایین باشد، ممکن است مشتری‌هایی که تقلب نکرده‌اند را به اشتباه متقلب شناسایی کنیم که باعث نارضایتی مشتری‌ها خواهد شد.

بازخوانی (recall):

نشان می دهد که از تمام موارد تقلب، چه کسری توسط مدل شناسایی شده اند. مقدار بالا یعنی مدل تقریباً همه نمونه های تقلب را شناسایی کرده، اما ممکن است برخی پیش بینی های نادرست داشته باشد. مقدار پایین یعنی بسیاری از نمونه های تقلب را از دست داده ایم.

این معیار از جهت شناسایی حداکثری موارد تقلب حائز اهمیت است.

F1-Score

از آنجایی که حداکثر کردن یکی از Recall یا Precision منجر به کمتر شدن دیگری میشود (مثلا اگر مدل به همه وردی ها خروجی ۱ بدهد، Recall حداکثر میشود ولی Precision به شدت افت میکند) برای همین معیار F1-Score به صورت میانگین هارمونیک این دو متغیر تعریف می شود. هر چقدر این معیار بیشتر باشد یعنی هر دوی Precision و Recall بالا تر بوده اند.

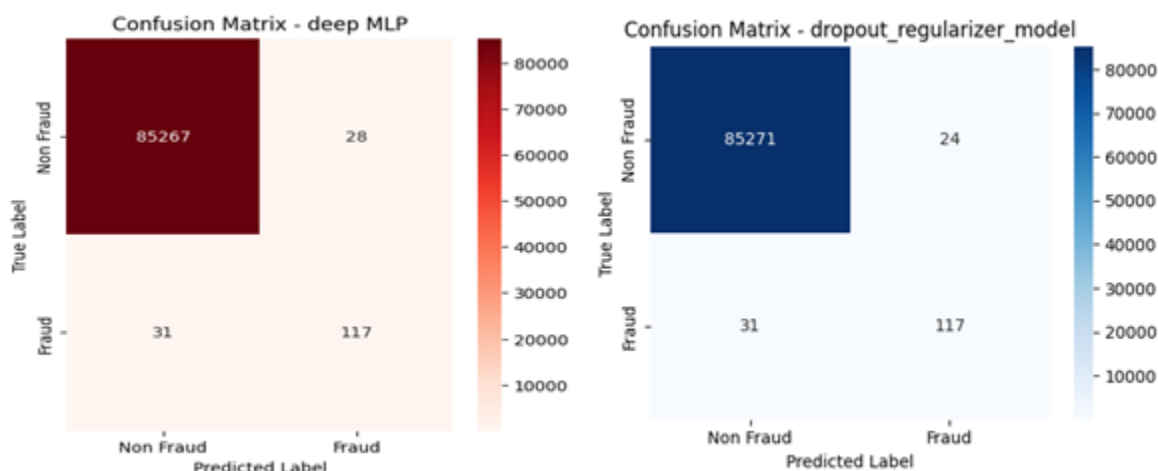
$$F1 - Score = 2 \frac{Precision * Recall}{Precision + Recall}$$

در داده های نامتوازن، دقت معمولی (accuracy) حتی اگر مدل به درستی کار نکند، می تواند مقدار بالایی را داشته باشد، به همین جهت استفاده از F1-Score معیار مناسب تری را برای ارزیابی مدل ارائه می دهد.

منحنی ROC و AUC

منحنی ROC نشان می دهد که مدل چقدر بین کلاس ها تفکیک ایجاد می کند. مقدار AUC (مساحت زیر نمودار) اگر به ۱ نزدیک باشد، یعنی مدل عملکرد خوبی دارد، اما اگر نزدیک به ۰.۵ باشد، مدل تقریباً تصادفی پیش بینی می کند. توضیحات بیشتر در این مورد در بخش های قبلی ارائه شده است.

ماتریس آشفستگی:



برای هر دو مدل FN بیشتر است، به این معناست که تمرکز مدل بر روی این بوده است که افرادی که مطمئناً متقلب بوده اند را شناسایی کند (Precision بالاتری داشته باشد) اما به طور کلی، چون بهترین مدل بر اساس F1-Score انتخاب شد، تعداد FN ها و FP ها تفاوت زیادی نداشتند.

تبادل (trade-off) بین دقت و بازخوانی:

افزایش Precision معمولاً باعث کاهش Recall می شود و برعکس. مثلاً اگر در تشخیص تقلب فقط مواردی را علامت بزنیم که کاملاً مطمئن هستیم (Precision بالا)، ممکن است بسیاری از موارد تقلب را از دست بدهیم (Recall پایین).

جست و جوی بهترین هایپرپارامترها شبکه یک لایه مخفی با Grid Search

Grid Search یک روش جستجو برای پیدا کردن بهترین ترکیب هایپرپارامترها در مدل های یادگیری ماشین است. در این روش، تمام ترکیب های ممکن از مقادیر داده شده بررسی شده و مدل برای هر ترکیب آموزش داده می شود. سپس، ترکیبی که بهترین عملکرد (مثلاً بالاترین دقت) را روی داده های اعتبارسنجی دارد، انتخاب می شود. این روش تضمین می کند که بهترین تنظیمات را پیدا کنیم، اما با افزایش تعداد پارامترها و مقادیر آنها، زمان محاسباتی زیادی نیاز دارد. این پیاده سازی در کد با استفاده از `itertools.product` انجام شده است تا تمام حالات ایجاد شوند و با توجه به عدم توازن داده ها بهترین مدل بر اساس F1-Score انتخاب شد.

مدل با تمامی ترکیب های پارامترهای زیر ایجاد و ارزیابی شده است:

`neurons_list = [۲۵۶, ۱۲۸, ۶۴]`

`dropout_list = [۰.۴, ۰.۳, ۰.۲]`

`regularization_list = [۰.۰۰۰۱, ۰.۰۰۰۱]`

`batch_sizes = [۶۴, ۳۲, ۱۶]`

نتیجه `grid search` در جدول زیر، که بر اساس F1-Score مرتب شده، قابل مشاهده است.

جدول ۲

F1-Score	Accuracy	Batch Size	Regularization λ	Dropout	تعداد نورون های لایه مخفی	مدل
0.83274	0.9994	16	0.001	0.3	64	0
0.83274	0.9994	64	0.0001	0.4	128	1
0.83274	0.9994	32	0.0001	0.3	256	2
0.831541	0.9994	32	0.001	0.4	64	3
0.831541	0.9994	32	0.001	0.3	256	4
0.830325	0.9994	64	0.001	0.3	128	5
0.829787	0.9994	64	0.0001	0.2	128	6
0.829787	0.9994	16	0.001	0.4	128	7
0.829787	0.9994	32	0.0001	0.2	256	8
0.829787	0.9994	64	0.001	0.3	256	9
0.827338	0.9994	64	0.0001	0.3	64	10
0.827338	0.9994	16	0.0001	0.3	128	11
0.827338	0.9994	32	0.0001	0.3	128	12
0.827338	0.9994	16	0.0001	0.2	256	13
0.826855	0.9994	32	0.0001	0.2	64	14
0.826087	0.9994	16	0.001	0.2	128	15
0.825623	0.9994	32	0.0001	0.4	64	16
0.825623	0.9994	64	0.0001	0.4	64	17
0.825279	0.9994	64	0.0001	0.2	256	18
0.824373	0.9994	16	0.001	0.2	64	19
0.824373	0.9994	32	0.001	0.3	128	20
0.824373	0.9994	64	0.001	0.4	128	21
0.824373	0.9994	64	0.0001	0.4	256	22
0.823105	0.9994	64	0.001	0.3	64	23
0.823105	0.9994	32	0.0001	0.4	256	24
0.822695	0.9994	16	0.0001	0.2	64	25
0.822695	0.9994	16	0.0001	0.3	256	26
0.821429	0.9994	32	0.001	0.3	64	27

0.821429	0.9994	16	0.001	0.3	128	28
0.819788	0.9994	64	0.0001	0.3	128	29
0.818841	0.9994	32	0.001	0.2	64	30
0.818841	0.9994	16	0.001	0.4	256	31
0.818505	0.9994	64	0.001	0.2	128	32
0.817518	0.9994	16	0.001	0.4	64	33
0.817204	0.9994	16	0.0001	0.2	128	34
0.817204	0.9994	32	0.0001	0.2	128	35
0.816901	0.9994	16	0.0001	0.4	128	36
0.815884	0.9994	16	0.001	0.3	256	37
0.815603	0.9994	64	0.001	0.2	64	38
0.810219	0.9994	32	0.001	0.2	256	39
0.808824	0.9994	32	0.0001	0.3	64	40
0.808824	0.9994	32	0.001	0.4	128	41
0.806084	0.9994	32	0.0001	0.4	128	42
0.804511	0.9994	64	0.0001	0.3	256	43
0.798611	0.9993	16	0.0001	0.4	64	44
0.797297	0.9993	32	0.001	0.2	128	45
0.797251	0.9993	64	0.001	0.2	256	46
0.795848	0.9993	16	0.0001	0.3	64	47
0.793103	0.9993	16	0.001	0.2	256	48
0.789855	0.9993	16	0.0001	0.4	256	49
0.771242	0.9992	64	0.001	0.4	256	50
0.770992	0.9993	64	0.001	0.4	64	51
0.763251	0.9992	64	0.0001	0.2	64	52
0.672269	0.9991	32	0.001	0.4	256	53

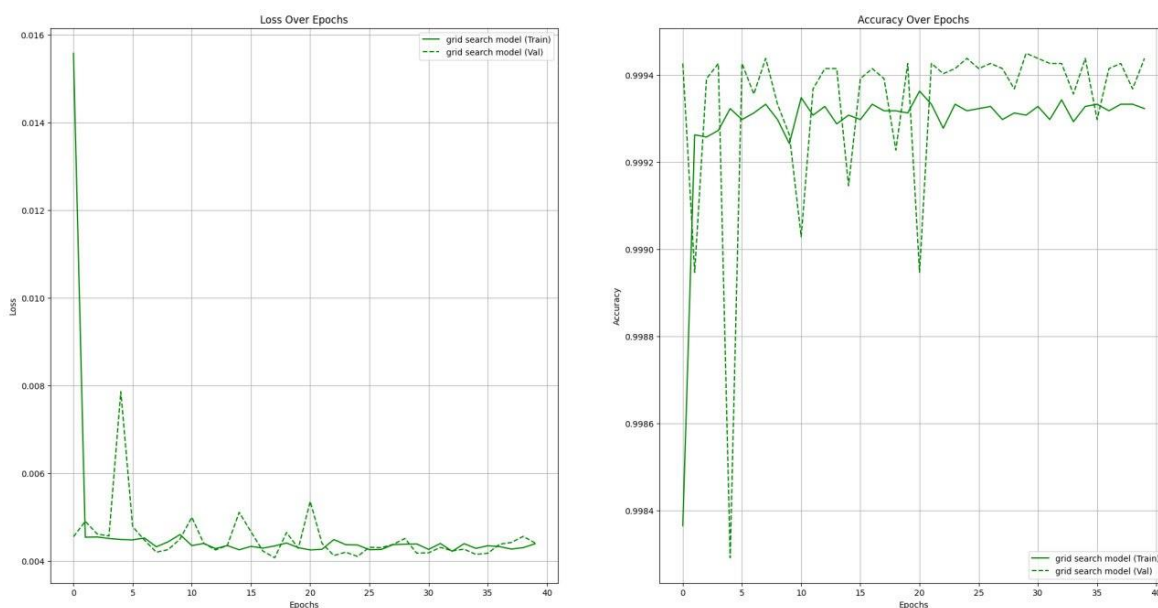
مدل های ۰، ۱ و ۲ F1-Score یکسانی دارند اما در میان آن ها مدل ابتدایی (مدل ۰) پیچیدگی کمتری دارد و به عنوان مناسبترین مدل انتخاب می شود.

بهترین مدل (F1-Score=0.83274) هایپر پارامتر های زیر را داراست:

Neurons=64, Dropout=0.3, Regularization=0.001, Batch Size=16, Accuracy=0.9994

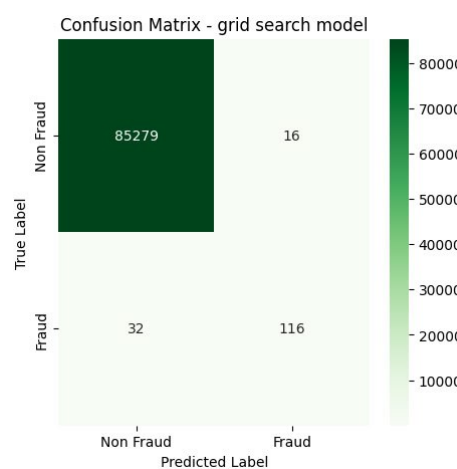
ارزیابی مدل:

نمودار Loss و Accuracy در طول ایپاک ها در شکل زیر دیده می شود. همانطور که پیش تر گفته شد، accuracy معیار خوبی برای ارزیابی این مدل نیست.



شکل ۲۳

ماتریس آشفتگی: در این مدل بر خلاف مدل های قبلی، تفاوت قابل توجهی میان FN و FP وجود دارد. همانطور که در شکل زیر دیده می شود، مدل در تشخیص کلاس non-fraud بهتر عمل می کند.



شکل ۲۴

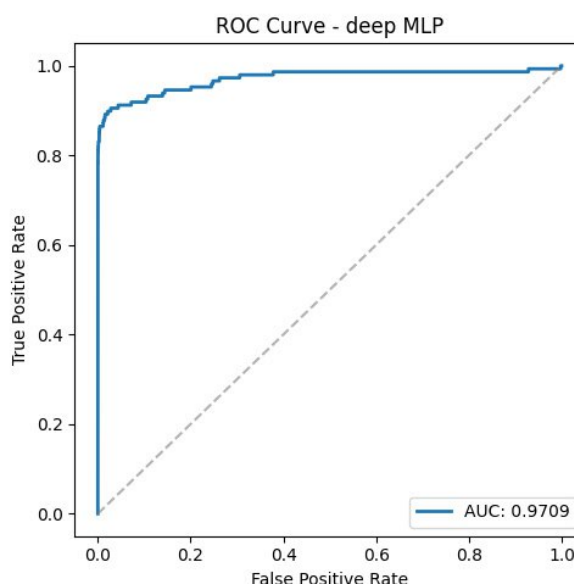
F1-Score و Accuracy, Recall, Precision

در شکل زیر مدل به دست آمده از grid search با مدل های قبلی مقایسه شده است. این مدل مقدار کمی نسبت به مدل dropout_regularizer بهبود داشته اما این تغییر قابل توجه نیست.

	Accuracy	Precision	Recall	F1-Score
plain_model	0.999391	0.863636	0.770270	0.814286
dropout_model	0.999438	0.931034	0.729730	0.818182
regularizer_model	0.999368	0.835714	0.790541	0.812500
dropout_regularizer_model	0.999415	0.871212	0.777027	0.821429
deep_model	0.999309	0.806897	0.790541	0.798635
grid_search_model	0.999438	0.878788	0.783784	0.828571

شکل ۲۵

نمودار ROC این مدل هم مشابه مدل های قبلی به دست آمده و AUC Score بالایی کسب کرده است.



شکل ۲۶

جمع‌بندی

- کدام مدل بهترین عملکرد را داشت و چرا؟

همانطور که انتظار میرفت مدل بدست آمده به کمک grid search بهترین نتیجه را داشت، چرا که عملاً از بین تعداد زیادی مدل مختلف بهترین را انتخاب کرده بودیم. اما جا دارد این نکته مجدداً ذکر شود که تفاوت این مدل با بهترین مدل مرحله اول (dropout + regularization) مقدار بسیار پایینی بود (در حدود ۰.۰۱ با معیار F1-Score) و در بین خود مدل‌هایی که توسط grid search هم بررسی شدند تعدادی مدل دقت یکسانی داشتند و مدلی که کمترین تعداد پارامترها (تعداد نوروں و ...) را داشت انتخاب شد.

- افزودن لایه‌های بیشتر چه تأثیری روی عملکرد مدل داشت؟

همانطور که در بالا هم اشاره شد مدل عمیق‌تر عملاً بهبودی‌ای نسبت به مدل‌های دیگر نداشت، این میتواند از این جهت باشد که افزودگی مورد نیاز توسط مدل‌های با عمق کمتر هم تأمین شده بود و افزودن آن عملاً تأثیری در عملکرد نداشت و صرفاً زمان آموزش را اضافه‌تر میکرد، هرچند ممکن است با انتخاب معماری‌های دیگر (مثلاً تعداد نوروں‌های متفاوت یا dropout متفاوت) بتوان مدل را بهبود داد ولی در مدلی که در صورت سوال ذکر شده بود بهبودی خاصی مشاهده نشد!

- بهینه‌سازی پارامترها چگونه بر نتیجه مدل تأثیر گذاشت؟

بهینه‌سازی پارامترهایی همچون تعداد نوروں‌ها و میزان dropout و... همانطور که از شکل ۲۵ مشخص است تا حدی در بهبود مدل تأثیر گذار بوده است و مدل‌نهایی که از grid search بدست آمد عملکرد بهتری نسبت به مدل اولیه داشت. همچنین باید توجه داشت که اگر این کار انجام نمیشد نمیتوانستیم جلوی overfit شدن مدلی اولیه را بگیریم و به کمک این پارامترها بود که مشکل overfit حل شد.

- میزان خطای مدل چقدر بود و علت اصلی آن چه بود؟

برای بررسی میزان خطای مدل، مدل‌نهایی که بهترین نتیجه را داشت (یعنی مدل بدست آمده از grid search) را معیار قرار میدهیم:

اگر منظور از میزان خطا میزان بر حسب accuracy باشد، میزان خطا ۰.۰۰۰۵۶۲ بود ولی همانطور که در بالا به کرات ذکر شد برای دیتاست‌های نامتوازن این معیار مناسب نیست.

به لحاظ F1-Score هم بهترین مدل عدد ۰.۸۲۸۵۷۱ را کسب کرد و recall و precision آن به ترتیب ۰.۷۸۳۷۸۴ و ۰.۸۷۸۷۸۸ بود.

- بر اساس جدول آشفستگی، مدل بیشتر در کدام کلاس ها اشتباه کرده است؟
با توجه به شکل ۲۴ مدل FN های بیشتری داشته که یعنی دقت آن در تشخیص درست کلاس های Fraud کمتر بوده. این امر میتواند از این ناشی باشد که تعداد اعضای کلاس Fraud کمتر بوده اند و مدل تعداد کمتری از آنها را برای یادگیری (نسبت که کلاس دیگر) دیده است.
- بین دقت و بازخوانی چه رابط های وجود دارد و چرا در تشخیص تقلب مهم است ؟
افزایش Precision معمولاً باعث کاهش Recall می شود و برعکس. مثلاً اگر در تشخیص تقلب فقط مواردی را علامت بزنیم که کاملاً مطمئن هستیم (Precision بالا)، ممکن است بسیاری از موارد تقلب را از دست بدهیم (Recall پایین). از طرف دیگر، اگر بخواهیم تمام موارد تقلب ممکن را شناسایی کنیم (Recall بالا)، احتمال دارد که برخی تراکنش های سالم را به اشتباه به عنوان تقلب در نظر بگیریم (Precision پایین). این تبادل بین Precision و Recall در سیستم های تشخیص تقلب بسیار مهم است، زیرا باید تعادلی برقرار شود که کمترین خطای مثبت کاذب (False Positive) و منفی کاذب (False Negative) ممکن را داشته باشیم. معمولاً از F1-score یا ROC-AUC برای یافتن بهترین نقطه تعادل بین این دو معیار استفاده می شود.
- مقایسه مدل ها: آیا مدل پیچیده تر عملکرد بهتری داشت یا فقط محاسبات را افزایش داد ؟
مدل های پیچیده تر از دو جهت بررسی شدند:
۱. مدل ها با لایه های بیشتر: همانطور که بالاتر ذکر شده است، افزودن لایه تاثیر خاصی در افزایش دقت نداشت.
۲. مدل با تعداد نورون های بالاتر: با توجه به نتایج به دست آمده از grid search مدل با حداقل تعداد نورون (۶۴) عملکرد یکسانی با مدل های با تعداد نورون های بالاتر داشت.
در نتیجه، پیچیده کردن مدل، تنها محاسبات را افزایش داد و بهبودی در عملکرد ایجاد نکرد.
- چه روش هایی برای بهبود عملکرد مدل پیشنهاد می شود ؟
با توجه به اینکه بهترین هایپر پارامتر ها توسط grid search به دست آمده بود و از روش هایی چون Dropout و regulariztion هم برای جلوگیری از بیش برازش به کار رفته بودند، ممکن است بتوان با افزایش داده ها یا متوازن کردن کلاس های داده ها عملکرد را بهبود داد.
- مهمترین چالش های تشخیص تقلب در داده های واقعی چیست؟

یکی از مهم‌ترین چالش‌های تشخیص تقلب در داده‌های واقعی، تعداد زیاد ویژگی‌ها و انتخاب ویژگی‌های مؤثر است. مشخص کردن اینکه کدام ویژگی‌ها بیشترین تأثیر را دارند و چه ویژگی‌های جدیدی باید ساخته شوند، نقش کلیدی در بهبود عملکرد مدل دارد. به‌عنوان مثال، در این مسئله آموزشی، زمان به‌صورت مطلق استفاده نشده است، درحالی‌که می‌توانست با تبدیل آن به ویژگی‌های مرتبط (مانند توزیع زمانی تراکنش‌ها یا تفاوت زمانی بین فعالیت‌ها) اطلاعات مفیدتری ارائه دهد. بنابراین، مهندسی ویژگی (Feature Engineering) و کاهش ابعاد داده از طریق روش‌هایی مانند تحلیل همبستگی و روش‌های انتخاب ویژگی، از چالش‌های اصلی در این حوزه هستند.

- اگر مدل را دوباره طراحی کنید، چه تغییری در آن ایجاد می‌کنید؟

با توجه به اینکه نمی‌توان تعداد داده‌ها را افزایش داد، با استفاده از روش‌هایی مثل oversampling داده‌ها را متوازن می‌کردیم و سپس مدل را بر روی داده‌های متوازن آموزش می‌دادیم.

پرسش ۲ - طراحی شبکه عصبی چندلایه در مسئله رگرسیون مقاومت بتن

مقدمه

مقاومت بتن یکی از مهم‌ترین ویژگی‌های آن است که تأثیر مستقیمی بر استحکام و دوام سازه‌های بتنی دارد. پیش‌بینی دقیق این مقاومت می‌تواند به بهینه‌سازی ترکیبات بتن و کاهش هزینه‌های ساخت کمک کند. با توجه به پیچیدگی روابط بین ترکیبات بتن و مقاومت نهایی آن، استفاده از روش‌های یادگیری ماشین، به‌ویژه شبکه‌های عصبی پرسپترون چندلایه (MLP)، می‌تواند رویکردی کارآمد برای مدل‌سازی این رابطه باشد.

هدف نهایی این تحقیق، ارائه مدلی بهینه برای پیش‌بینی مقاومت بتن و بررسی نحوه تأثیر تغییرات تنظیمات مدل بر دقت پیش‌بینی است. نتایج به‌دست‌آمده می‌تواند به مهندسان عمران و محققان صنعت ساختمان کمک کند تا با استفاده از مدل‌های یادگیری ماشین، ترکیبات بتن را بهینه‌سازی کرده و کیفیت سازه‌ها را افزایش دهند.

آماده سازی دادگان و تحلیل آماری

برای انجام این بخش، از مجموعه داده Concrete Strength استفاده شده است که شامل اطلاعاتی درباره ترکیبات بتن و مقاومت آن است. این داده‌ها از Kaggle دریافت شده و شامل ۱۰۳۰ نمونه با ۹ ویژگی (همگی از جنس عدد) هستند.

	CementComponent	BlastFurnaceSlag	FlyAshComponent	WaterComponent	SuperplasticizerComponent	CoarseAggregateComponent	FineAggregateComponent	AgeInDays	Strength
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.99
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.89
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.27
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.05
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.30
...	...	...	...	...	...	...	...	...	...
1025	276.4	116.0	90.3	179.6	8.9	870.1	768.3	28	44.28
1026	322.2	0.0	115.6	196.0	10.4	817.9	813.4	28	31.18
1027	148.5	139.4	108.6	192.7	6.1	892.4	780.0	28	23.70
1028	159.1	186.7	0.0	175.6	11.3	989.6	788.9	28	32.77
1029	260.9	100.5	78.3	200.6	8.6	864.5	761.5	28	32.40

1030 rows x 9 columns

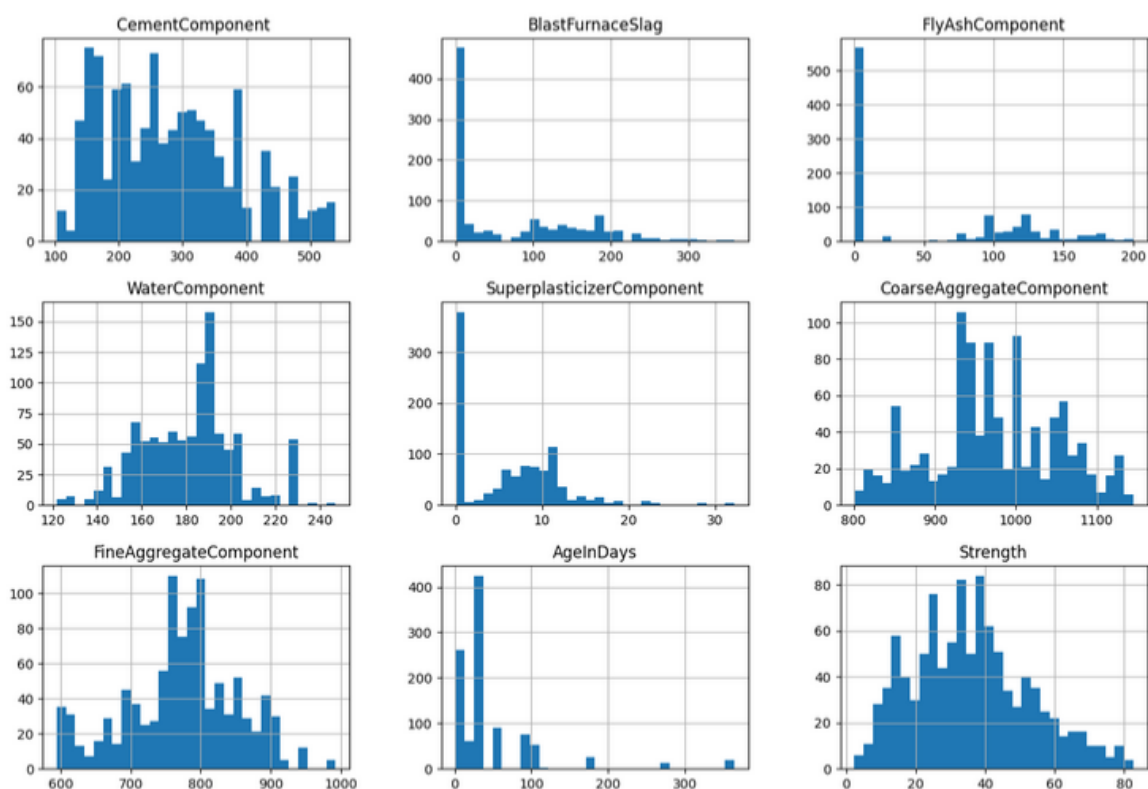
شکل ۲۷

جدول زیر خلاصه‌ای از ویژگی‌های آماری مجموعه داده را نشان می‌دهد، از جمله مقدار میانگین، انحراف معیار، حداقل و حداکثر مقادیر هر ویژگی.

	CementComponent	BlastFurnaceSlag	FlyAshComponent	WaterComponent	SuperplasticizerComponent	CoarseAggregateComponent	FineAggregateComponent	AgeInDays	Strength
count	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000
mean	281.167864	73.895825	54.188350	181.567282	6.204660	972.918932	773.580485	45.662136	35.817961
std	104.506364	86.279342	63.997004	21.354219	5.973841	77.753954	80.175980	63.169912	16.705742
min	102.000000	0.000000	0.000000	121.800000	0.000000	801.000000	594.000000	1.000000	2.330000
25%	192.375000	0.000000	0.000000	164.900000	0.000000	932.000000	730.950000	7.000000	23.710000
50%	272.900000	22.000000	0.000000	185.000000	6.400000	968.000000	779.500000	28.000000	34.445000
75%	350.000000	142.950000	118.300000	192.000000	10.200000	1029.400000	824.000000	56.000000	46.135000
max	540.000000	359.400000	200.100000	247.000000	32.200000	1145.000000	992.600000	365.000000	82.600000

شکل ۲۸

برای اینکه درک بهتری از نمونه ها داشته باشیم، هیستوگرام مربوط به هر ویژگی را رسم می کنیم:



شکل ۲۹

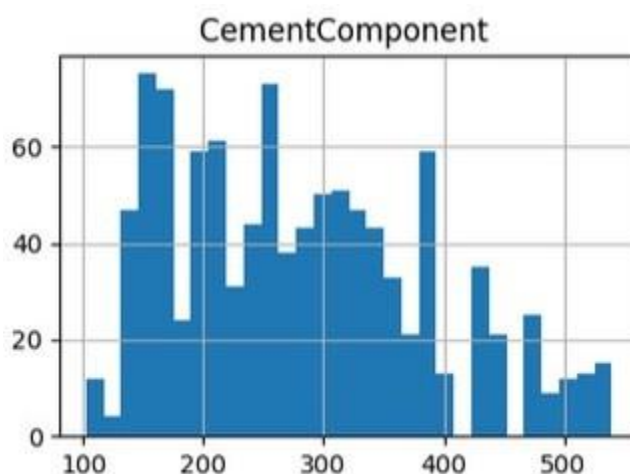
در ادامه به تحلیل هیستوگرام می پردازیم.

۱. CementComponent (مقدار سیمان)

داده‌ها در بازه‌ای بین حدود ۱۰۰ تا ۵۰۰ قرار دارند.

بیشترین نمونه‌ها در مقدار ۲۰۰ تا ۳۵۰ توزیع شده‌اند.

این توزیع دارای چندین قله است که نشان‌دهنده استفاده از ترکیب‌های متفاوت سیمانی در نمونه‌های بتن است.



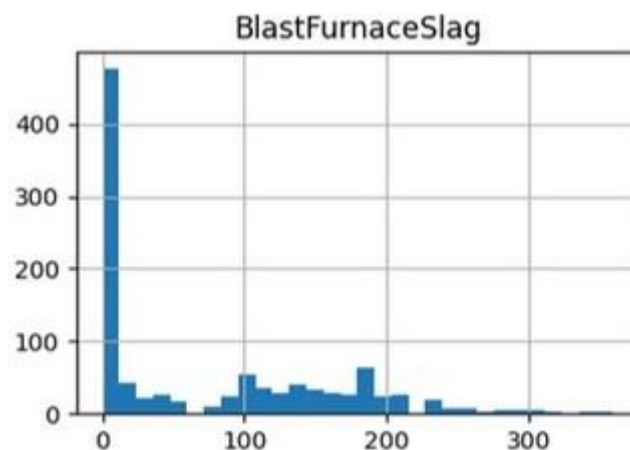
شکل ۳۰

۲. BlastFurnaceSlag (سرباره کوره بلند)

مقدار این ویژگی در بسیاری از نمونه‌ها نزدیک به صفر است.

تنها در تعداد کمی از نمونه‌ها از مقدار قابل توجهی سرباره استفاده شده است.

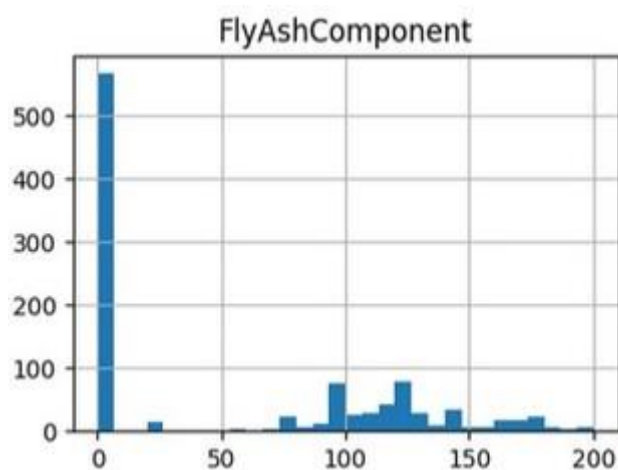
این توزیع نشان می‌دهد که در بسیاری از موارد، سیمان به‌جای سرباره در ترکیب بتن استفاده شده است.



شکل ۳۱

۳. FlyAshComponent (خاکستر بادی)

الگوی مشابهی با BlastFurnaceSlag دارد؛ بیشتر نمونه‌ها مقدار بسیار کمی از این ماده را دارند. برخی از نمونه‌ها از مقادیر بالاتری از Fly Ash استفاده کرده‌اند، اما این موارد در اقلیت هستند. نشان می‌دهد که در اکثر ترکیبات، از خاکستر بادی به مقدار کم استفاده شده است.

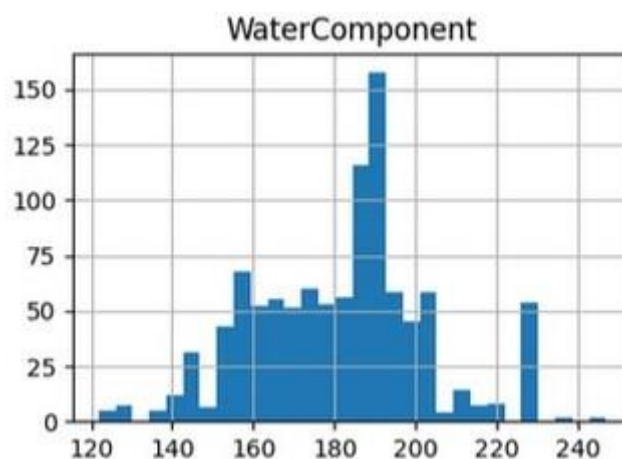


شکل ۳۲

۴. WaterComponent (میزان آب)

داده‌ها بیشتر در بازه ۱۵۰ تا ۲۰۰ متمرکز هستند.

یک توزیع نسبتاً متقارن دارد، که نشان‌دهنده استفاده پایدار از میزان آب در اکثر نمونه‌هاست.



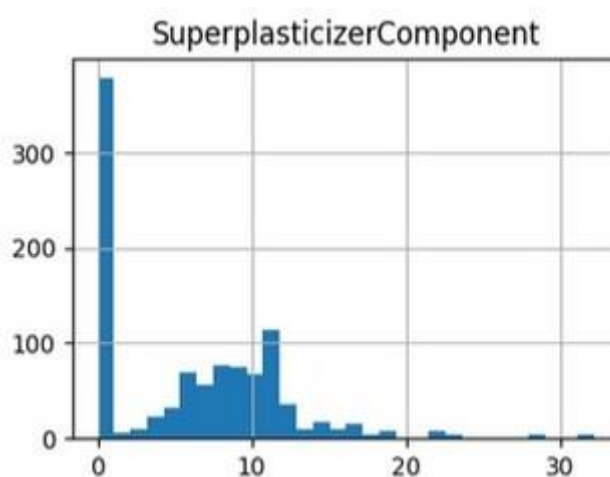
شکل ۳۳

۵. SuperplasticizerComponent (فوق روان‌کننده)

داده‌ها به شدت چوله هستند، به طوری که در بیشتر موارد مقدار این ویژگی صفر یا نزدیک به صفر است.

در تعداد کمی از نمونه‌ها از مقادیر بالاتر Superplasticizer استفاده شده است.

این توزیع نشان می‌دهد که اکثر نمونه‌های بتن بدون فوق روان‌کننده تهیه شده‌اند.

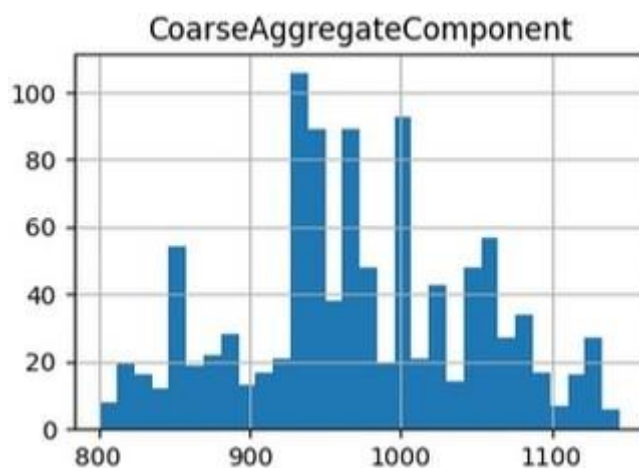


شکل ۳۴

۶. CoarseAggregateComponent (سنگدانه درشت)

داده‌ها در بازه ۸۰۰ تا ۱۱۰۰ پراکنده هستند.

دارای چندین قله است که نشان‌دهنده استفاده از چندین ترکیب استاندارد برای این ماده در ساخت بتن است.

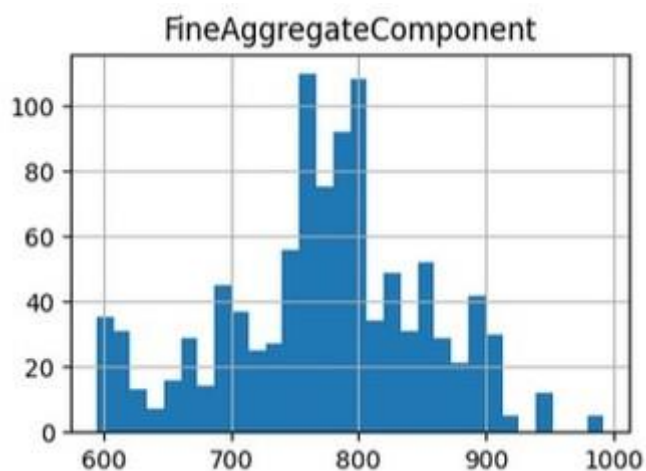


شکل ۳۵

۷. FineAggregateComponent (سنگدانه ریز)

در بازه ۶۰۰ تا ۱۰۰۰ قرار دارد.

توزیع نسبتاً متقارنی دارد که نشان می‌دهد مقدار سنگدانه ریز در بیشتر نمونه‌ها تقریباً در مقدار مشخصی ثابت است.



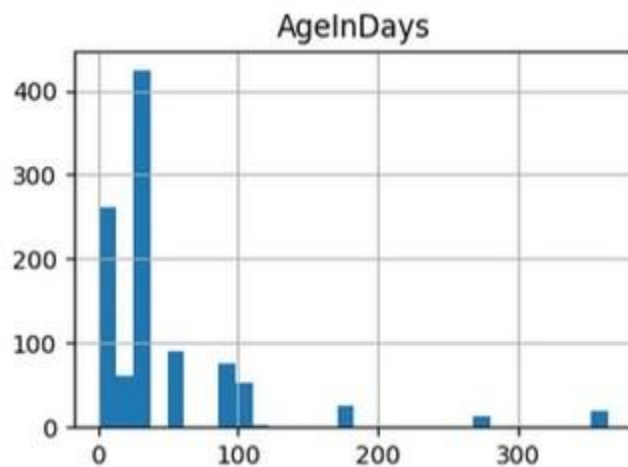
شکل ۳۶

۸. AgeInDays (سن بتن بر حسب روز)

مقدار این ویژگی به شدت چوله است؛ بیشتر داده‌ها در مقادیر پایین (کمتر از ۵۰ روز) متمرکز شده‌اند.

تعداد کمی از نمونه‌ها دارای سن بسیار بالایی هستند.

این الگو نشان می‌دهد که بیشتر نمونه‌های آزمایش‌شده، در سنین پایین مورد بررسی قرار گرفته‌اند.



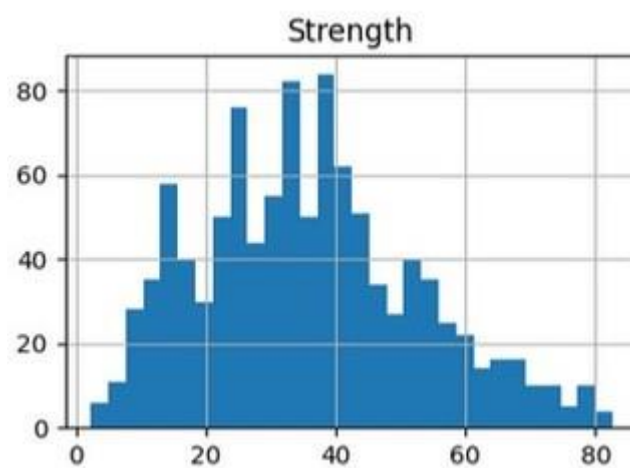
شکل ۳۷

۹. Strength (مقاومت فشاری بتن)

این ویژگی دارای توزیع تقریباً نرمال با مقدار بیشینه در محدوده ۲۰ تا ۴۰ مگاپاسکال است.

چند نمونه دارای مقدار مقاومت بالاتر از ۶۰ مگاپاسکال هستند، اما این موارد کمتر دیده می‌شوند.

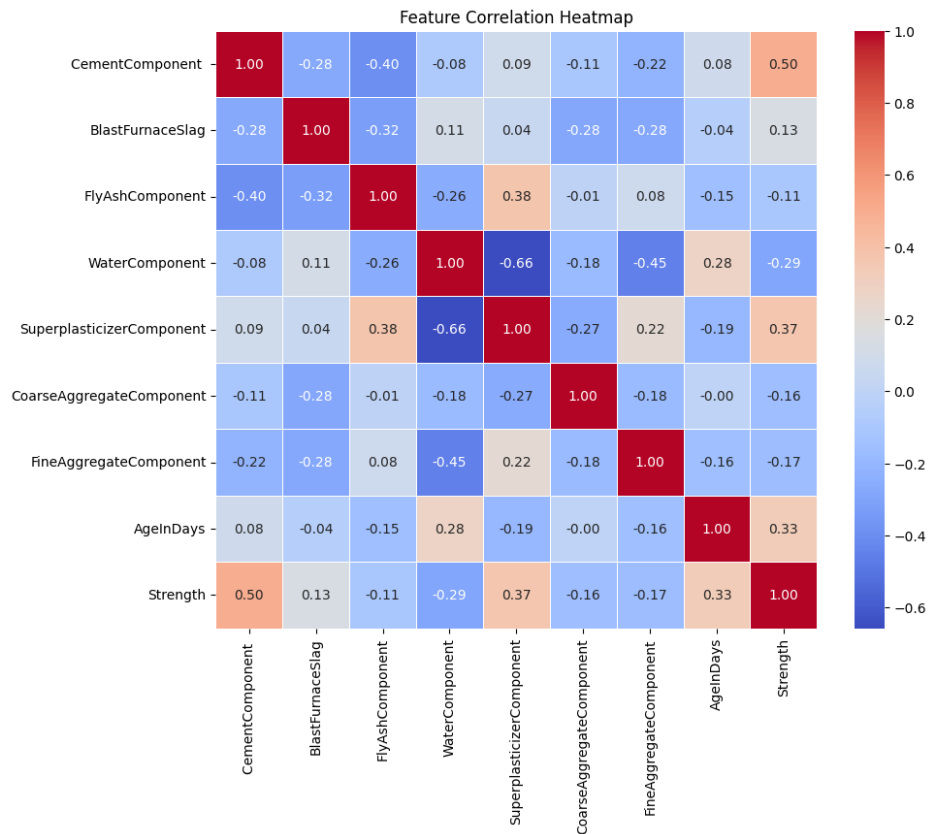
این الگو نشان می‌دهد که بیشتر نمونه‌های بتنی دارای مقاومت متوسط بوده و تعداد کمی از نمونه‌ها به مقاومت بسیار بالا رسیده‌اند.



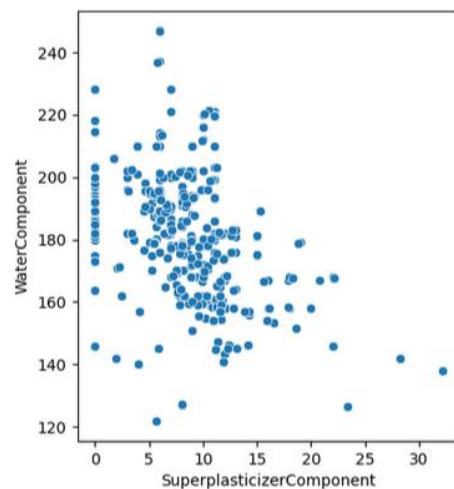
شکل ۳۸

بررسی همبستگی ویژگی‌ها با یکدیگر

ماتریس همبستگی نشان می‌دهد که اکثر ویژگی‌ها با یکدیگر همبستگی زیادی ندارند. تنها بین دو ویژگی SuperplasticizerComponent و WaterComponent همبستگی نسبتاً قوی ۰/۶۶- مشاهده می‌شود. نمودار پراکندگی این دو ویژگی نسبت به یکدیگر در ادامه آمده است.



شکل ۳۹



شکل ۴۰

حذف ویژگی‌های با همبستگی کم:

تأثیر مثبت:

- اگر ویژگی واقعاً بی‌ارتباط با خروجی باشد، حذف آن باعث کاهش نویز و بهبود تعمیم‌پذیری مدل می‌شود.
- مدل سبک‌تر و سریع‌تر اجرا می‌شود.

تأثیر منفی:

- ممکن است برخی ویژگی‌های کم‌اهمیت به صورت ترکیبی مفید باشند، بنابراین حذف تک‌تک آن‌ها می‌تواند به کاهش عملکرد منجر شود.
- مدل ممکن است اطلاعات مفیدی را که به طور غیرمستقیم تأثیر دارند، از دست بدهد.

حذف ویژگی‌های با همبستگی زیاد:

تأثیر مثبت:

- اگر دو ویژگی به شدت همبسته باشند، یکی از آن‌ها اطلاعات جدیدی اضافه نمی‌کند و ممکن است این دو ویژگی عملاً یکسان باشند. حذف یکی باعث کاهش ابعاد داده و جلوگیری از اثر دادن یک ویژگی از حد واقعی می‌شود.
- مدل با داده‌های کمتر، سریع‌تر و پایدارتر یاد می‌گیرد.

تأثیر منفی:

- در برخی موارد، حذف ویژگی اشتباه ممکن است اطلاعات مهمی را از بین ببرد و پیش‌بینی را ضعیف کند.
- در یادگیری عمیق، ویژگی‌های همبسته ممکن است به صورت ترکیبی مؤثر باشند، حذف یکی ممکن است تأثیر منفی داشته باشد.

حذف ویژگی‌های بسیار همبسته معمولاً به بهبود مدل کمک می‌کند، اما باید آزمایش شود که کدام ویژگی را حذف کنیم. همچنین بهتر است به جای حذف کامل چند ویژگی با همبستگی بالا، با استفاده از روش‌هایی مانند PCA، ویژگی جدیدی را جایگزین کنیم.

بررسی همبستگی ویژگی‌ها با مقاومت بتن

بررسی همبستگی ویژگی‌ها با مقاومت بتن به ما درک بهتری از تأثیر هر متغیر بر خروجی مدل می‌دهد. با تحلیل این روابط، می‌توان ویژگی‌های تأثیرگذار را شناسایی کرده و مدل را بهینه‌سازی کرد. همچنین، همبستگی بالا بین برخی ویژگی‌ها ممکن است نشان‌دهنده وجود اطلاعات تکراری باشد که می‌تواند بر عملکرد مدل تأثیر بگذارد. در نتیجه، این بررسی به انتخاب ویژگی‌های مناسب و کاهش پیچیدگی مدل کمک می‌کند.

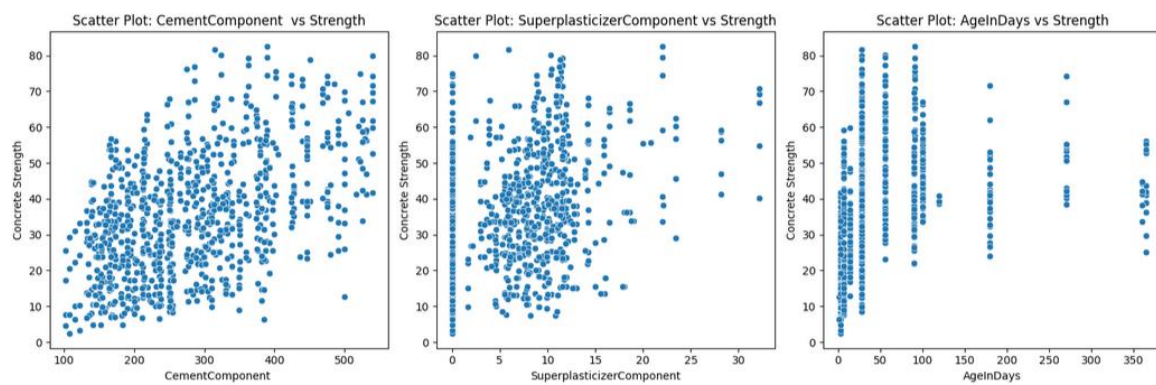
همانطور که مشاهده می‌شود به ترتیب CementComponent، SuperplasticizerComponent و AgeInDays بیشترین همبستگی را با مقاومت بتن دارند.

```
correlation_target = corr_matrix["Strength"]\
    .sort_values(ascending=False, key=np.abs)
print(correlation_target)
```

```
Strength          1.000000
CementComponent   0.497832
SuperplasticizerComponent 0.366079
AgeInDays         0.328873
WaterComponent    -0.289633
FineAggregateComponent -0.167241
CoarseAggregateComponent -0.164935
BlastFurnaceSlag   0.134829
FlyAshComponent    -0.105755
Name: Strength, dtype: float64
```

شکل ۴۱

برای درک بهتر این همبستگی، نمودار پراکندگی این ویژگی‌ها نسبت به مقاومت بتن در ادامه آورده شده است:



شکل ۴۲

پیاده سازی مدل شبکه عصبی چند لایه

مقاومت بتن (Strength) را به عنوان لیبل کلاس‌ها (y) از مجموعه داده جدا می‌کنیم و سپس مجموعه داده را به train و test تفکیک می‌کنیم. این کار به ما امکان می‌دهد مدل را بر روی داده‌های آموزشی یاد بگیریم و عملکرد آن را بر روی داده‌های آزمایشی ارزیابی کنیم. همچنین ورودی را نرمالایز می‌کنیم تا ارزش ویژگی‌ها در ابتدا یکسان شود.

با توجه به اینکه در ادامه مدل با لایه‌های مخفی مختلف، تعداد epoch متفاوت، و بهینه‌سازهای گوناگون نیاز است، تابع create_model نوشته شده است تا یک مدل شبکه عصبی چندلایه با قابلیت تنظیم تعداد نرون‌های لایه مخفی، نوع بهینه‌ساز و تابع هزینه ایجاد کند.

```
def create_model(hidden_neurons=16, optimizer='adam', loss='mse'):
    model = Sequential([
        Dense(hidden_neurons, activation='relu', input_shape=(X_train.shape[1],)),
        Dense(1, activation='linear')
    ])

    model.compile(optimizer=optimizer, loss=loss, metrics=['mse', 'mae', tf.keras.losses.Huber()])
    return model

def fit(model, X_train, y_train, X_test, y_test, epochs):
    return model.fit(X_train, y_train, epochs=epochs, validation_data=(X_test, y_test), verbose=0)

def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    return {
        'mse': mean_squared_error(y_test, y_pred),
        'mae': mean_absolute_error(y_test, y_pred)
    }
```

شکل ۴۳

مدل شامل یک لایه مخفی با تعداد نرون‌های مشخص و تابع فعال‌ساز ReLU است.

خروجی مدل یک مقدار عددی (مقاومت بتن) است، بنابراین تابع فعال‌ساز خروجی خطی (Linear) در نظر گرفته شده است.

Sequential یک مدل ترتیبی در Keras است که به ما امکان می‌دهد لایه‌های شبکه عصبی را به‌صورت پیاپی تعریف کنیم. در این روش، هر لایه دقیقاً به لایه بعدی متصل می‌شود.

Dense یک لایه کاملاً متصل (Fully Connected Layer) است که در آن هر نرون با تمام نرون‌های لایه قبلی ارتباط دارد.

مدل با بهینه‌سازهای مختلف و تابع هزینه قابل تنظیم آموزش داده می‌شود. علاوه بر آن، معیارهای MSE (میانگین مربعات خطا)، MAE (میانگین قدر مطلق خطا) و Huber Loss نیز برای ارزیابی عملکرد مدل استفاده شده‌اند.

پس از ساخت مدل، تابع fit برای آموزش مدل با تعداد epoch مشخص و مجموعه داده‌های آموزشی و آزمایشی تعریف شده است. این تابع مدل را آموزش داده و میزان خطای روی مجموعه اعتبارسنجی را نیز ثبت می‌کند.

در نهایت، تابع evaluate برای محاسبه و گزارش میزان خطای مدل روی داده‌های آزمایشی استفاده می‌شود. این تابع خروجی مدل را محاسبه کرده و میزان MSE و MAE را برای ارزیابی دقت مدل برمی‌گرداند.

ساخت مدل اولیه با ۱۶ و ۳۲ لایه مخفی

دو مدل اولیه با ۱۶، ۳۲ لایه مخفی و این مشخصات مشترک می‌سازیم و بر روی داده‌های train آموزش (fit) می‌دهیم.

- تعداد اپیاک: ۵۰
- بهینه ساز: adam
- تابع زیان: MSE

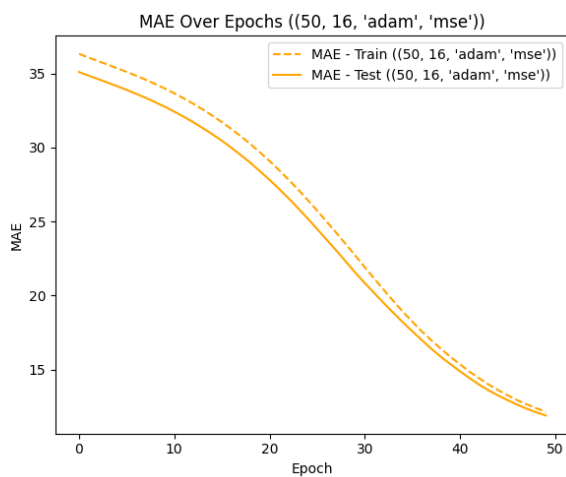
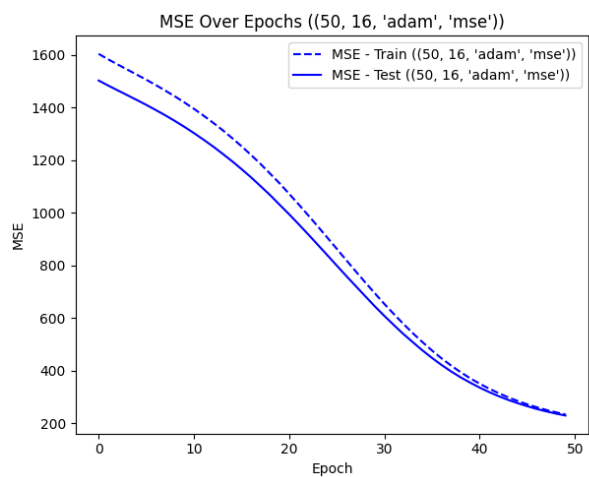
نتایج مقایسه این دو مدل با لایه های مخفی متفاوت به صورت زیر است:

```
16 hidden layers: {'mse': 199.25124741687864, 'mae': 11.302497707971863}  
32 hidden layers: {'mse': 159.15054508843613, 'mae': 10.142139686411639}
```

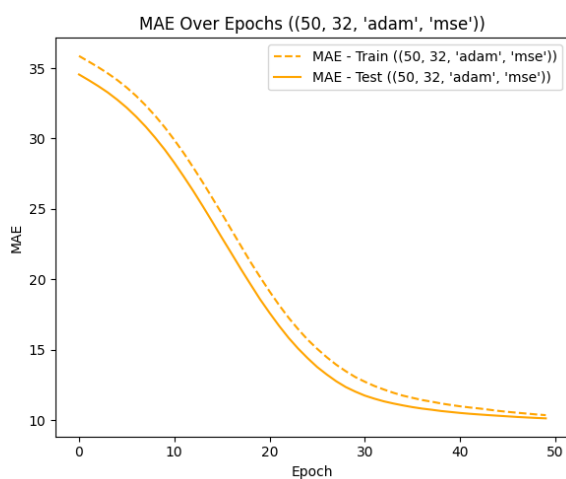
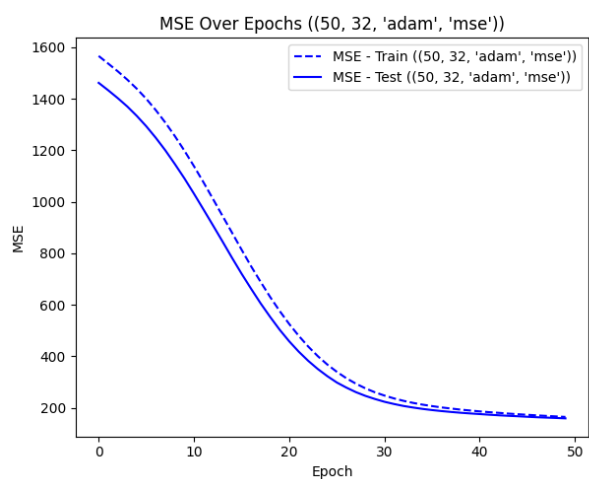
شکل ۴۴

مدل با ۳۲ لایه عملکرد نهایی بهتری نسبت به ۱۶ لایه دارد.

افزایش تعداد نوروں‌ها از ۱۶ به ۳۲ باعث بهبود عملکرد مدل می‌شود، زیرا ظرفیت یادگیری آن افزایش یافته و می‌تواند الگوهای پیچیده‌تری را شناسایی کند. این باعث کاهش خطای آموزش و بهبود تعمیم مدل می‌شود، البته تا جایی که از بیش‌برازش جلوگیری شود.



شکل ۴۵

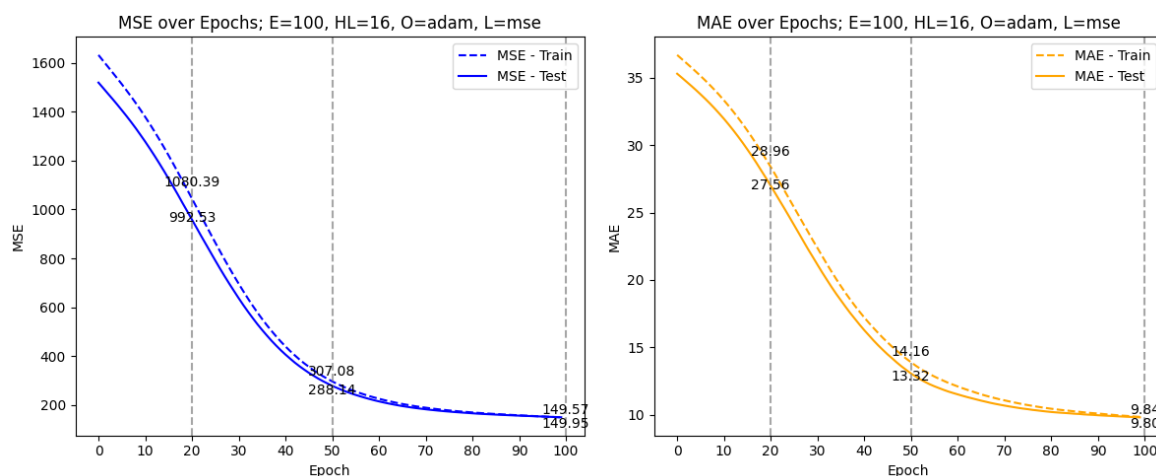


شکل ۴۶

بررسی تغییرات تنظیمات مدل

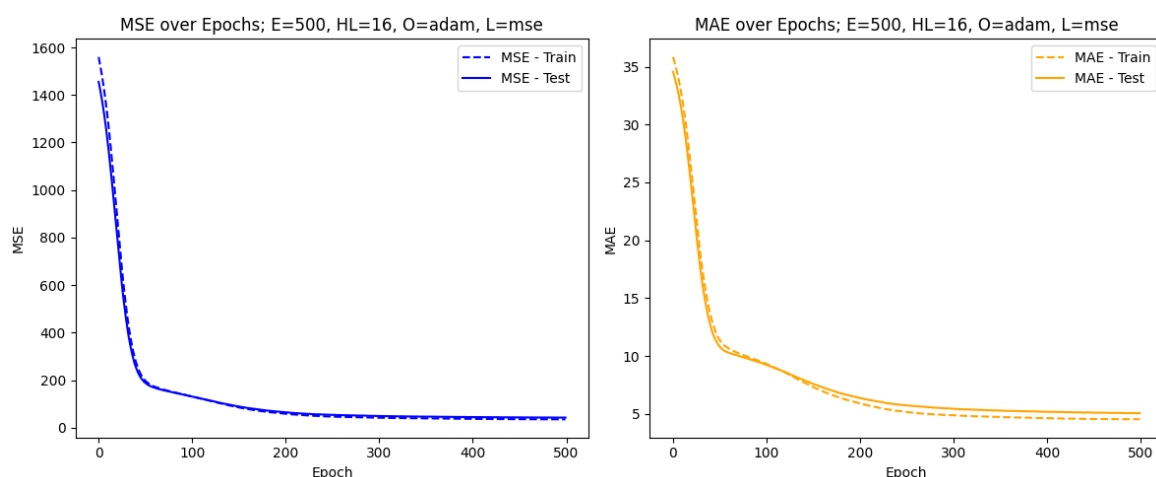
تأثیر ایپاک:

نتیجه دقت بر روی داده های تست (مدل با در ۲۰، ۵۰ و ۱۰۰ ایپاک)



شکل ۴۷

افزایش تعداد ایپاکها تا حدی می تواند دقت مدل را بهبود ببخشد، زیرا مدل فرصت بیشتری برای یادگیری الگوهای موجود در داده ها دارد. اما پس از یک نقطه مشخص، ادامه آموزش ممکن است منجر به بیش برآش (Overfitting) شود، جایی که مدل به جای یادگیری الگوهای کلی، جزئیات و نویز داده های آموزش را حفظ می کند. در این حالت، خطای داده های آموزش کاهش می یابد، اما عملکرد مدل روی داده های تست افت می کند و خطا در test به train همگرا نمی شود.



شکل ۴۸

از طرفی، ایپاک های بیشتر زمان بیشتری برای train کردن نیاز دارد و در مواردی بهبود محسوسی رخ نمی دهد. همانطور که تفاوت MAE در ۵۰ ایپاک و ۱۰۰ ایپاک جزئی است.

تأثیر توابع هزینه:

در این بخش MSE، MAE و Huber Loss به عنوان توابع هزینه (زیان) مقایسه می شوند. ابتدا به

اختصار هر کدام از این توابع را بررسی می کنیم:

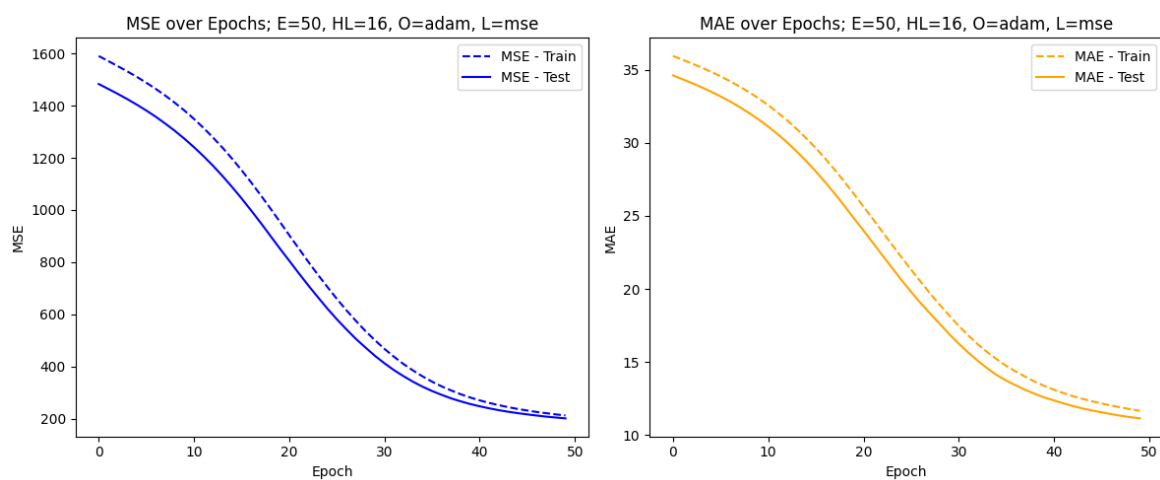
- MSE یا میانگین مربعات خطا مقدار اختلاف بین پیش‌بینی مدل و مقدار واقعی را به توان دو می‌رساند و سپس میانگین می‌گیرد. این تابع به خطاهای بزرگ حساس‌تر است و در صورت وجود نقاط پرت، مقدار آن به شدت افزایش می‌یابد.
- MAE یا میانگین قدر مطلق خطا اختلاف بین پیش‌بینی و مقدار واقعی را بدون به توان رساندن محاسبه می‌کند. این تابع در مقایسه با MSE تأثیر نقاط پرت را کمتر می‌کند، اما گرادیان آن در بهینه‌سازی کمتر هموار است.
- Huber Loss ترکیبی از MSE و MAE است. این تابع در صورت کوچک‌بودن خطاها مانند MSE رفتار می‌کند و در صورت بزرگ‌بودن خطاها مانند MAE عمل می‌کند؛ به عبارت دیگر، Huber Loss از مزیت‌های هر دو تابع MSE و MAE بهره می‌برد و در عین حال معایب آن‌ها را کاهش می‌دهد. این تابع برای مسئله‌های رگرسیون که در آن‌ها خطاهای بزرگ نادر ولی مهم هستند مناسب است. فرمول محاسبه این تابع به این صورت است:

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \leq \delta, \\ \delta \cdot (|a| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases}$$

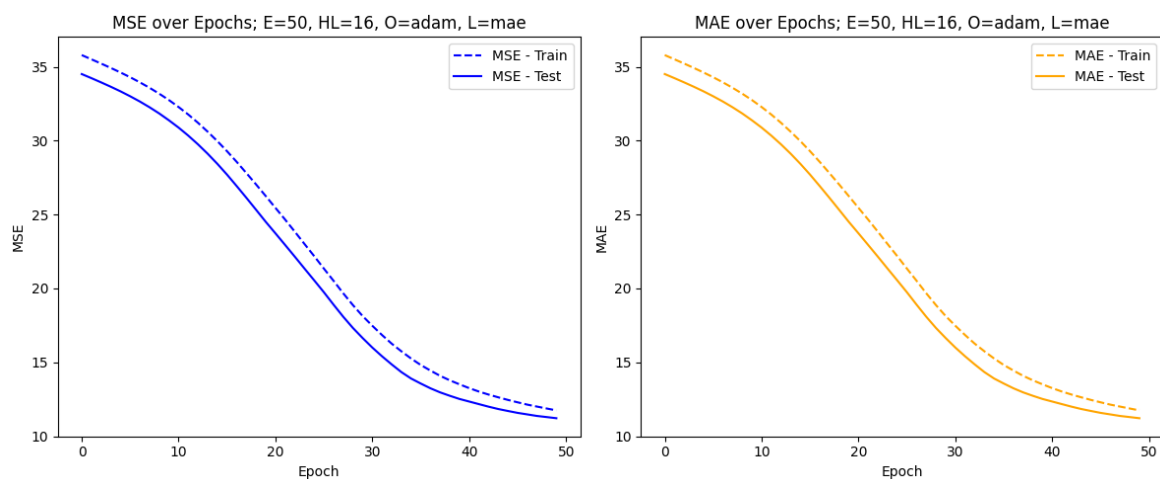
شکل ۴۹

δ پارامتر آستانه (threshold) این تابع است. هر چقدر این مقدار بزرگ‌تر باشد، Huber Loss برای مقادیر بیشتری مشابه با MSE عمل می‌کند. در ابتدا این مقدار را برابر با ۱ قرار می‌دهیم.

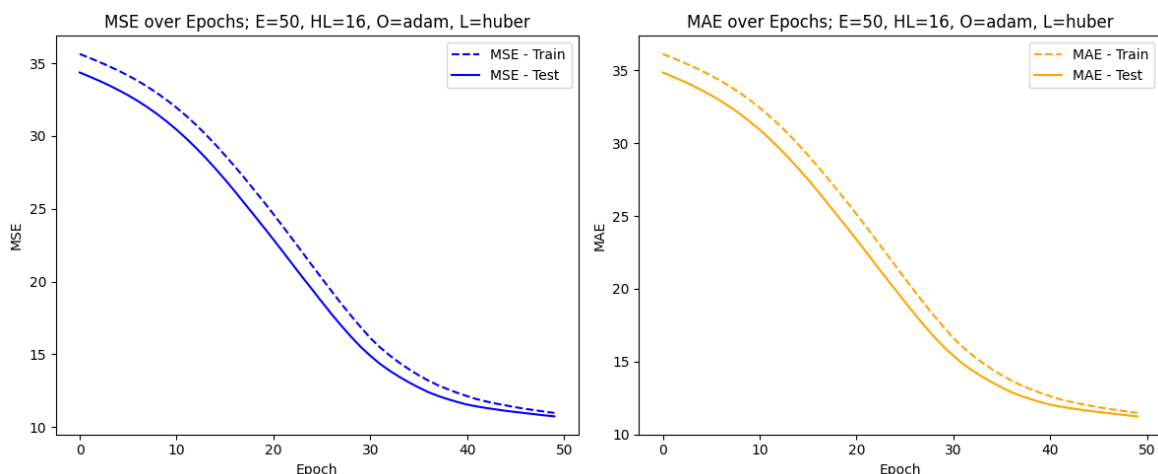
با مقایسه این سه تابع هزینه برای مدل متوجه می شویم که Huber Loss نسبت به MSE و MAE عملکرد بهتری دارد، اما این تفاوت نسبت به MAE ناچیز است.



شکل ۵۰



شکل ۵۱



شکل ۵۲

با تغییر پارامتر دلتا ممکن است به نتایج بهتری نیز برسیم. برای اینکار چند پارامتر مختلف را بررسی می کنیم:

```
Delta: 0.01, Final Validation Loss: {'mse': 201.3409470471985, 'mae': 11.100419148812403}
Delta: 0.05, Final Validation Loss: {'mse': 222.96303656776084, 'mae': 11.558175503382019}
Delta: 0.1, Final Validation Loss: {'mse': 232.39917188276357, 'mae': 11.986950047563965}
Delta: 0.5, Final Validation Loss: {'mse': 201.4823590162634, 'mae': 11.08459501099818}
Delta: 1.0, Final Validation Loss: {'mse': 195.7118603638687, 'mae': 11.125654945620441}
Delta: 2.0, Final Validation Loss: {'mse': 230.63151630434868, 'mae': 11.827895006840283}
Delta: 5.0, Final Validation Loss: {'mse': 198.56866074605762, 'mae': 11.167334698673976}
Delta: 10.0, Final Validation Loss: {'mse': 205.4162289734283, 'mae': 11.454151913911394}
```

شکل ۵۳

همانطور که مشاهده می شود، با تغییر پارامتر دلتا تفاوت قابل توجهی ایجاد نمی شود و این مدل با اینکه نسبت به MAE بهتر است، اما این تفاوت برای این داده ها جزئی است.

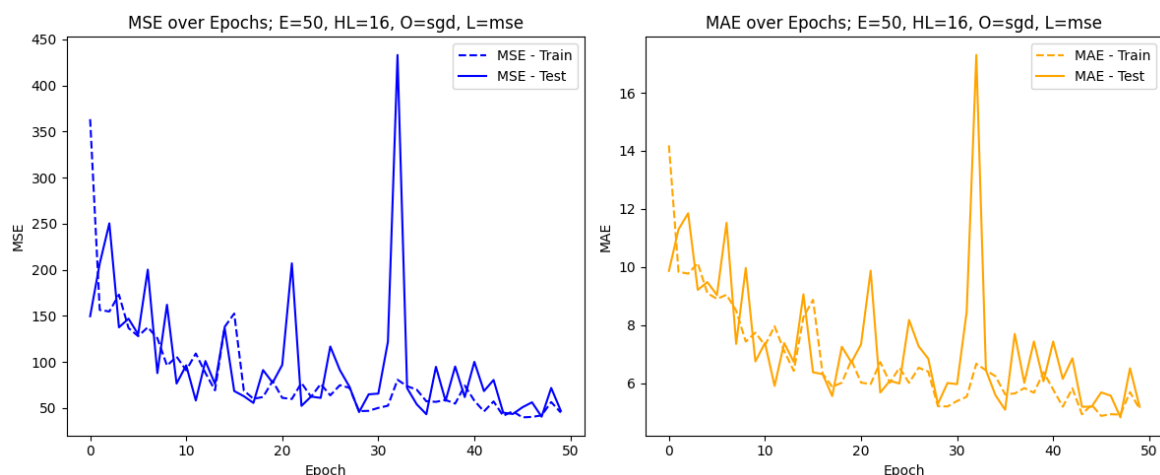
تأثیر توابع بهینه ساز:

توابع بهینه ساز (Optimizers) الگوریتم هایی هستند که در فرآیند آموزش مدل های یادگیری ماشین، وزن ها و بایاس ها را بر اساس مقدار گرادیان تابع هزینه به روزرسانی می کنند تا مدل به کمترین مقدار خطا برسد.

در شبکه های عصبی، هدف این است که خطا (Loss) را در طول اپیاک ها کاهش دهیم و توابع بهینه ساز این کار را با تنظیم میزان تغییر وزن ها در هر مرحله انجام می دهند. بهینه سازها تعیین می کنند

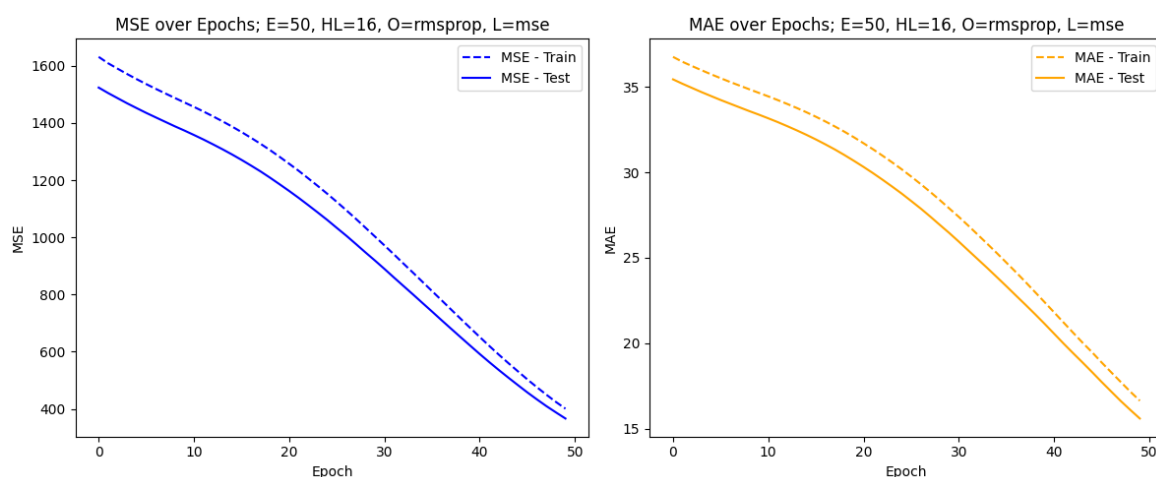
که مدل چقدر سریع و چگونه به حداقل تابع هزینه همگرا شود. در این بخش ما سه تابع SGD، RMSprop، و Adam را برای مدل استفاده کرده و مقایسه می کنیم.

SGD (Stochastic Gradient Descent) یک روش پایه برای بهینه سازی است که وزن ها را در هر مرحله با استفاده از گرادیان داده های تصادفی به روزرسانی می کند. این روش ساده و کارآمد است اما ممکن است در مینیمم محلی گیر کند یا نوسانات زیادی داشته باشد.



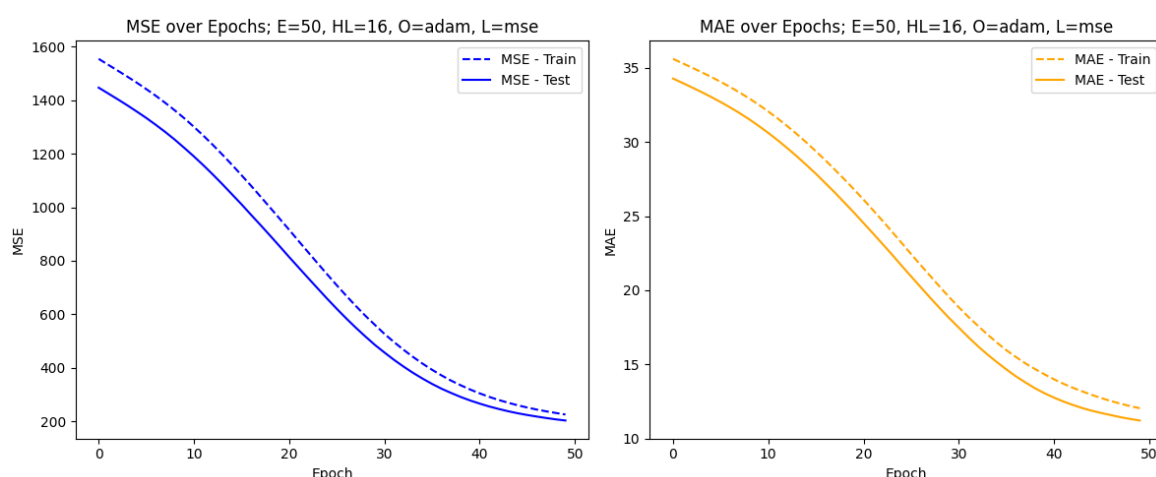
شکل ۵۴

RMSprop (Root Mean Square Propagation) نسخه بهبودیافته ای از SGD است که با استفاده از میانگین مربع گرادیان های گذشته، نرخ یادگیری را برای هر پارامتر به طور جداگانه تنظیم می کند. این روش سرعت همگرایی را افزایش داده و در حل مسائل دارای داده های نامتوازن یا noisy عملکرد بهتری دارد.



شکل ۵۵

Adam (Adaptive Moment Estimation) ترکیبی از RMSprop و Momentum است که هم نرخ یادگیری تطبیقی و هم سرعت بالای همگرایی را ارائه می‌دهد. این روش معمولاً عملکرد خوبی در انواع مختلف شبکه‌های عصبی دارد و به همین دلیل یکی از پرکاربردترین بهینه‌سازها است.



شکل ۵۶

همانطور که مشاهده می‌شود خطای نهایی SGD نسبت به سایر بهینه‌سازها کمتر است و عملکرد بهتری دارد. اما نرخ همگرایی آن نسبت به Adam و RMSprop پایین‌تر است، به این معنا که به تعداد ایپاک‌های بیشتری برای رسیدن به مقدار بهینه نیاز دارد. علاوه بر این، SGD مستعد نوسانات زیاد در مسیر کاهش خطا است، به‌خصوص در مسائل با داده‌های پیچیده یا دارای نویز. در مقابل، بهینه‌سازهای Adam و RMSprop به دلیل تنظیم نرخ یادگیری تطبیقی، سرعت همگرایی بالاتری دارند و تغییرات وزن‌ها را به شکل پایدارتری کنترل می‌کنند. بنابراین، اگرچه SGD در نهایت ممکن است به مقدار خطای کمتری برسد، اما روش‌های تطبیقی در برخی موارد می‌توانند سریع‌تر و کارآمدتر عمل کنند.

جمع بندی

- کدام ویژگی بیشترین تأثیر را روی مقاومت بتن دارد؟ آیا این ویژگی منطقی است؟

نتایج تحلیل همبستگی نشان می‌دهد که CementComponent بیشترین تأثیر را روی مقاومت بتن دارد. این نتیجه از نظر علمی نیز منطقی است، زیرا سیمان نقش اصلی را در ایجاد چسبندگی و استحکام بتن ایفا می‌کند. مقدار بیشتر سیمان معمولاً باعث افزایش مقاومت فشاری می‌شود، زیرا هیدراتاسیون سیمان منجر به تشکیل ساختار متراکم‌تر و قوی‌تری در بتن می‌شود. با این حال، میزان بهینه سیمان باید در نظر گرفته شود، زیرا افزایش بیش‌ازحد آن ممکن است منجر به جمع‌شدگی، ترک‌خوردگی و کاهش دوام بتن در بلندمدت شود.

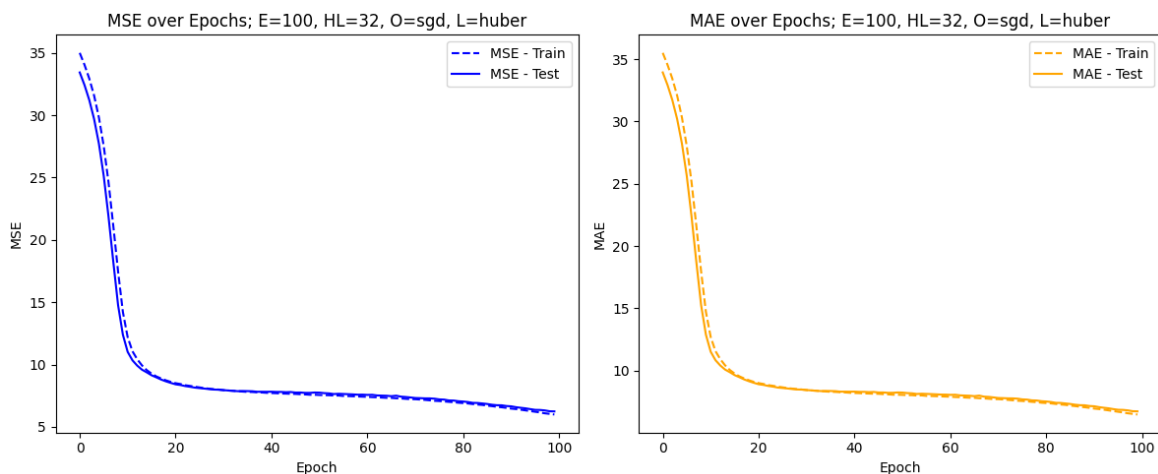
- بهترین تنظیمات برای این مدل چه بود؟

تعداد نورون‌های لایه مخفی: ۳۲

ایپاک: ۱۰۰

تابع هزینه: Huber Loss با $\delta=1$

تابع بهینه‌ساز: SGD



شکل ۵۷

- آیا افزایش تعداد ایپاک همیشه باعث بهبود مدل شد؟

بله؛ افزایش تعداد ایپاک‌ها تا حدی می‌تواند دقت مدل را بهبود ببخشد، زیرا مدل فرصت بیشتری برای یادگیری الگوهای موجود در داده‌ها دارد. اما پس از یک نقطه مشخص، ادامه آموزش ممکن است منجر به بیش‌برازش

(Overfitting) شود، جایی که مدل به جای یادگیری الگوهای کلی، جزئیات و نویز داده‌های آموزش را حفظ می‌کند. در این مسئله تا ۵۰۰ اپیاک بررسی شد و همواره test به train همگرا شد.

- کدام تابع هزینه دقت بیشتری داشت؟ چرا؟

Huber Loss؛ که ترکیبی از MSE و MAE است. این تابع در صورت کوچک بودن خطاها مانند MSE رفتار می‌کند و در صورت بزرگ بودن خطاها مانند MAE عمل می‌کند؛ به عبارت دیگر، Huber Loss از مزیت‌های هر دو تابع MSE و MAE بهره می‌برد و در عین حال معایب آن‌ها را کاهش می‌دهد.

- بین بهینه سازها، کدام سریعتر همگرا شد؟

مدل با بهینه ساز Adam سریعتر همگرا شد اما خطای نهایی sgd پایینتر بود.

- مهمترین چالش‌های مدلسازی رگرسیون با شبکه عصبی چه بودند؟

یافتن معماری بهینه شبکه: انتخاب تعداد لایه های مخفی و نورون ها در هر لایه برای رسیدن به تعادل بین قدرت یادگیری و جلوگیری از بیش برازش، چالش برانگیز بود.

انتخاب تابع هزینه و بهینه ساز: با توجه به ماهیت داده ها و هدف مدل سازی، باید تابع هزینه (Loss Function) و بهینه ساز (Optimizer) مناسب انتخاب شود.

تنظیم پارامتر دلتا در تابع هزینه Huber: در صورت استفاده از تابع هزینه Huber، تنظیم پارامتر دلتا برای بهینه سازی عملکرد مدل ضروری است.

پرسش ۳. پیاده سازی Adaline برای دیتاست IRIS

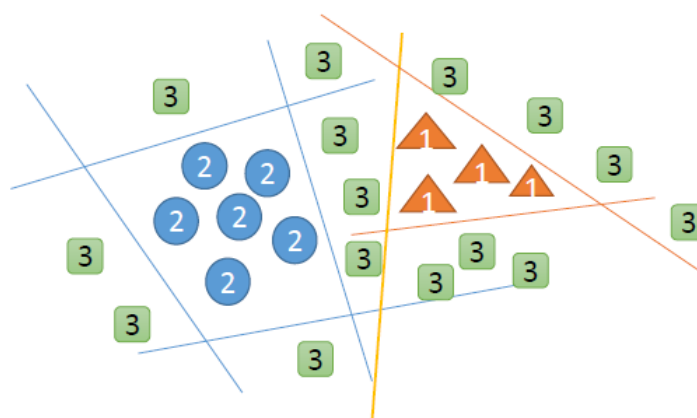
مقدمه

در این بخش، الگوریتم Adaline (Adaptive Linear Neuron) برای دسته‌بندی داده‌های Iris پیاده‌سازی شده است. این مدل یک شبکه عصبی تک‌لایه است که از یادگیری گرادیانی و تابع هزینه میانگین مربعات خطا (MSE) برای به‌روزرسانی وزن‌ها استفاده می‌کند. هدف از این آزمایش، بررسی عملکرد Adaline در جداسازی دو کلاس Setosa و Versicolor و تحلیل تأثیر نرخ یادگیری و تعداد اپیاک‌ها بر دقت مدل است.

آشنایی با Adaline

Adaline (Adaptive Linear Neuron): یک مدل پرسپترون تک‌لایه است که از مجموع خطی ورودی‌ها و یک تابع هزینه مبتنی بر MSE برای تنظیم وزن‌ها استفاده می‌کند. برخلاف پرسپترون معمولی، قبل از اعمال تابع آستانه (activation function)، وزن‌ها را با گرادیان نزولی به‌روزرسانی می‌کند.

Madaline (Multiple Adaline): نسخه توسعه‌یافته Adaline با چندین لایه Adaline است که در لایه اول چندین واحد Adaline قرار دارد و در لایه خروجی از منطق رأی‌گیری برای تصمیم‌گیری نهایی استفاده می‌شود. هنگامی که جداسازی داده‌ها تنها با یک خط قابل انجام نیست، می‌توان از Madaline بهره گرفت.



شکل ۵۸

تفاوت MLP و Adaline:

Adaline تک‌لایه و خطی است، درحالی‌که MLP (Multi-Layer Perceptron) دارای لایه‌های مخفی و غیرخطی است که از توابع فعال‌سازی مانند ReLU برای یادگیری روابط پیچیده استفاده می‌کند. MLP می‌تواند مسائل غیرخطی را حل کند، اما Adaline فقط برای مدل‌های خطی مناسب است. به طور خلاصه:

- Adaline یک شبکه عصبی تک‌لایه با تابع فعال‌سازی خطی است.
- Madaline یک شبکه عصبی چندلایه با تابع فعال‌سازی خطی است.
- MLP یک شبکه عصبی چندلایه با تابع فعال‌سازی غیرخطی است.

آماده‌سازی دادگان

- مجموعه داده: ابتدا مجموعه داده Iris با استفاده از تابع `load_iris` از کتابخانه `sklearn.datasets` بارگیری می‌شود.
- ایجاد `DataFrame`: داده‌های لود شده به یک `DataFrame Pandas` تبدیل می‌شوند تا کار با آنها آسان‌تر شود. ستون 'target' برای نشان دادن نوع گل (Setosa, Versicolor یا Virginica) به `DataFrame` اضافه می‌شود.
- حذف Virginica: برای سادگی و تمرکز بر طبقه‌بندی دودویی، داده‌های مربوط به نوع گل Virginica از `DataFrame` حذف می‌شوند.
- انتخاب ویژگی‌ها: فقط دو ویژگی "petal length (cm)" و "petal width (cm)" برای آموزش مدل Adaline انتخاب می‌شوند.
- تبدیل برچسب‌ها: برچسب‌های هدف (y) به مقادیر ۱- (برای Setosa) و ۰ (برای Versicolor) تبدیل می‌شوند تا با تابع فعال‌سازی Adaline سازگار باشند.
- نرمال‌سازی داده‌ها: ویژگی‌های انتخاب شده را نرمال‌سازی می‌کنیم. این کار باعث می‌شود که مقادیر ویژگی‌ها در محدوده [۰, ۱] قرار گیرند و به بهبود عملکرد مدل کمک می‌کند.

```
X_normalized = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
X_normalized[:10]

array([[0.09756098, 0.05882353],
       [0.09756098, 0.05882353],
       [0.07317073, 0.05882353],
       [0.12195122, 0.05882353],
       [0.09756098, 0.05882353],
       [0.17073171, 0.17647059],
       [0.09756098, 0.11764706],
       [0.12195122, 0.05882353],
       [0.09756098, 0.05882353],
       [0.12195122, 0.         ]])
```

شکل ۵۹

- تقسیم داده ها به train و test: داده ها با استفاده از تابع train_test_split از کتابخانه sklearn.model_selection به دو مجموعه آموزشی و آزمایشی تقسیم می شوند. داده ها به نسبت مساوی در کلاس ها قرار دارند، در نتیجه نیازی به stratified sampling نیست. همچنین چون داده ها به ترتیب کلاس در جدول داده قرار دارند، این تقسیم، باید تصادفی انجام شود.

```
print(len(y[y == -1]), len(y[y == 1]))

50 50
```

شکل ۶۰

پیاده سازی و آموزش مدل Adaline

به این منظور، کلاسی به نام Adaline تعریف کردیم:

```
class Adaline:
    def __init__(self, input_dimension, learning_rate=0.01, epochs=50):
        self.input_dimension = input_dimension
        self.alpha = learning_rate
        self.epochs = epochs

        self.weights = np.random.rand(input_dimension) * 0.1
        self.bias = np.random.rand() * 0.1

        self.losses = []
        self accuracies = []
        self.weights_history = []

    def activation(self, z):
        return z

    def fit(self, X, y, verbose=False):
        n_samples = X.shape[0]

        for epoch in range(self.epochs):
            linear_output = np.dot(X, self.weights) + self.bias
            y_pred = self.activation(linear_output)

            errors = y - y_pred
            self.weights += self.alpha * np.dot(X.T, errors) / n_samples
            self.bias += self.alpha * np.mean(errors)

            self.weights_history.append((self.weights.copy(), self.bias))

            mse = np.mean(errors**2)
            self.losses.append(mse)

            y_pred_class = np.where(y_pred >= 0, 1, -1)
            accuracy = np.mean(y_pred_class == y) * 100
            self accuracies.append(accuracy)

            if verbose:
                print(f"Epoch {epoch + 1}/{self.epochs}, Loss: {mse:.4f}, Accuracy: {accuracy:.2f}%")

    def predict(self, X):
        return np.where(self.activation(np.dot(X, self.weights) + self.bias) >= 0, 1, -1)
```

شکل ۶۱

مقداردهی اولیه:

در تابع `__init__`، پارامترهای مدل مانند نرخ یادگیری (`learning_rate`)، تعداد دوره های آموزشی (`epochs`) و بعد ورودی (`input_dimension`) تنظیم می شوند. همچنین، وزن ها و بایاس به طور تصادفی با استفاده از تابع `np.random.rand` مقداردهی اولیه می شوند. این مقداردهی اولیه تصادفی به مدل کمک می کند تا از نقاط بهینه محلی فرار کند و به یک راه حل کلی تر برسد.

آموزش:

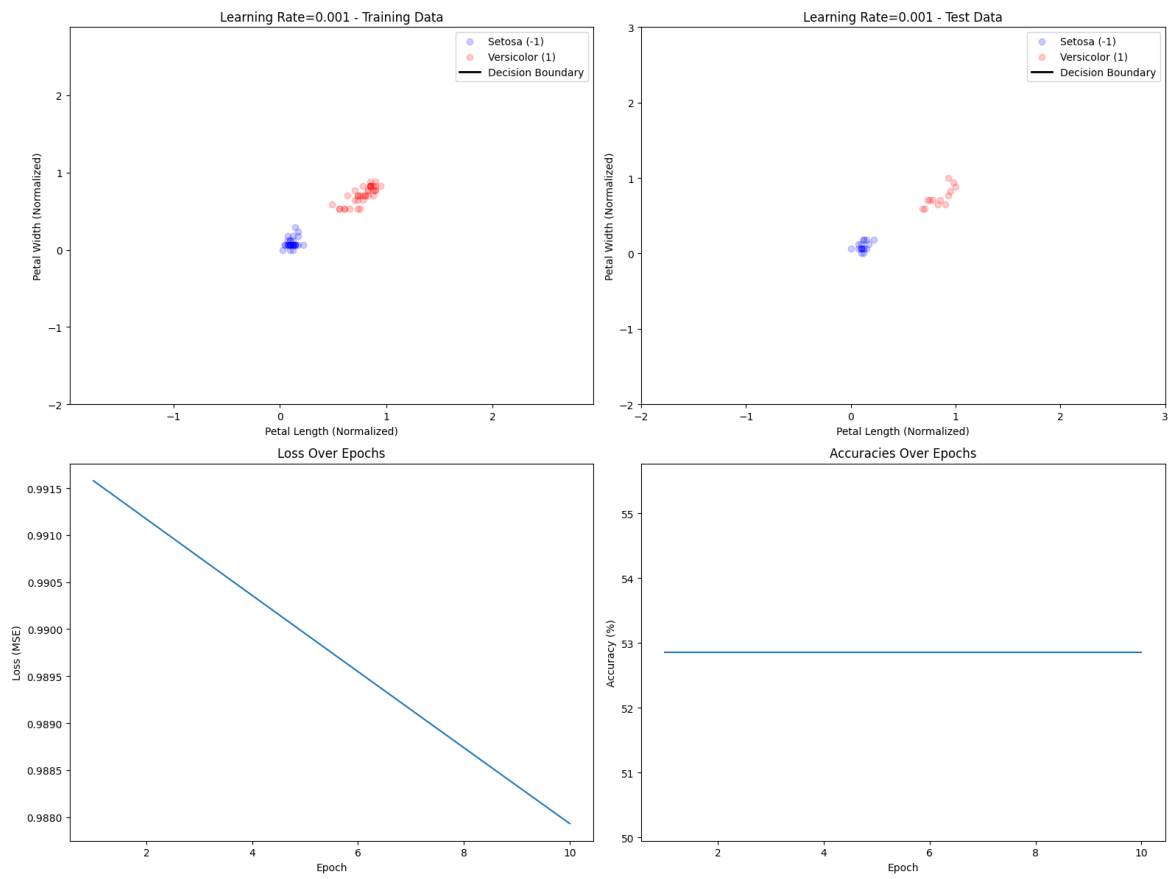
تابع `fit` برای آموزش مدل استفاده می شود. در هر دوره آموزشی، خروجی خطی مدل با استفاده از وزن ها و بایاس محاسبه می شود. سپس، خطای پیش بینی با استفاده از تابع فعال سازی (در اینجا تابع همانی) محاسبه می شود و تاریخچه ای از وزن ها، خطا و دقت در طول `epoch` ها نگهداری می شود. با استفاده از قانون دلتا، وزن ها و بایاس به روز می شوند تا خطا به حداقل برسد.

پیش بینی:

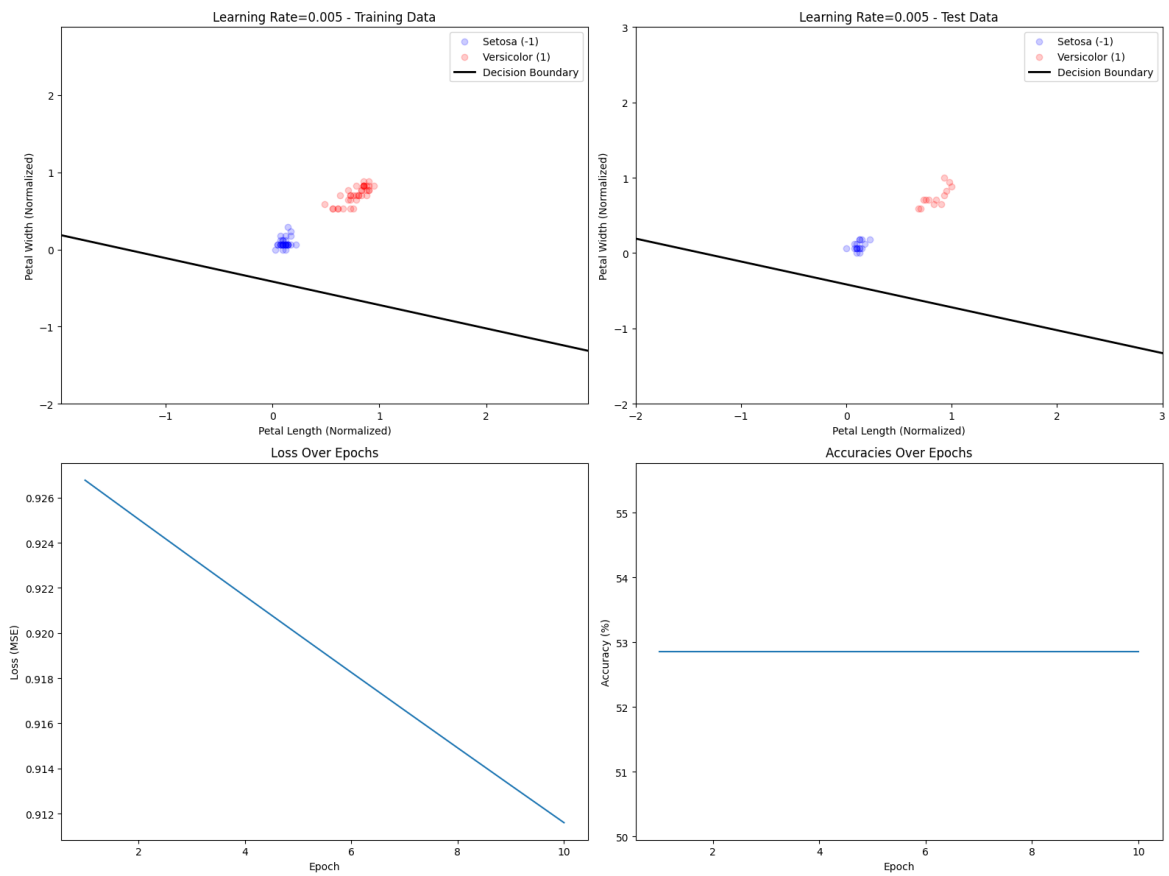
تابع `predict` برای پیش بینی برچسب کلاس برای داده های جدید استفاده می شود. خروجی خطی مدل محاسبه شده و با استفاده از تابع فعال سازی (در اینجا تابع همانی) به برچسب کلاس تبدیل می شود.

ساخت مدل ها:

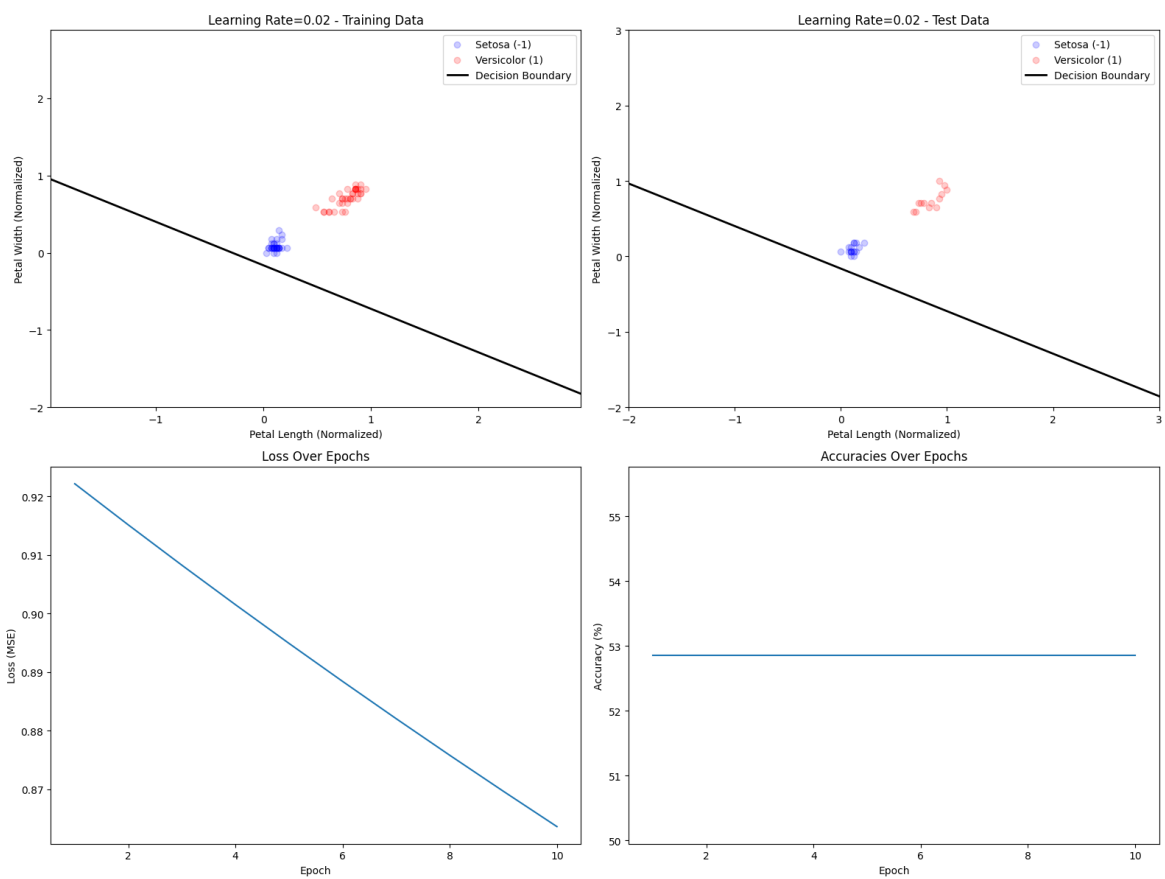
در شرح پروژه ذکر شده است که مدل با ۱۰ ایپاک و نرخ یادگیری های ۰.۰۰۱، ۰.۰۰۵ و ۰.۰۲ آموزش داده شود. اما پس از ساخت مدل ها متوجه شدیم اگر چه تابع زیان کاهش داده می شود، اما هیچ کدامشان تفکیک را به درستی انجام نمی دهند. دقت حدود ۵۰٪ در مسائل دسته بندی دودویی، هنگامی که کلاس ها متوازن هستند، تفاوت چندانی با انتخاب تصادفی و برچسب زدن ندارد.



شکل ۶۲



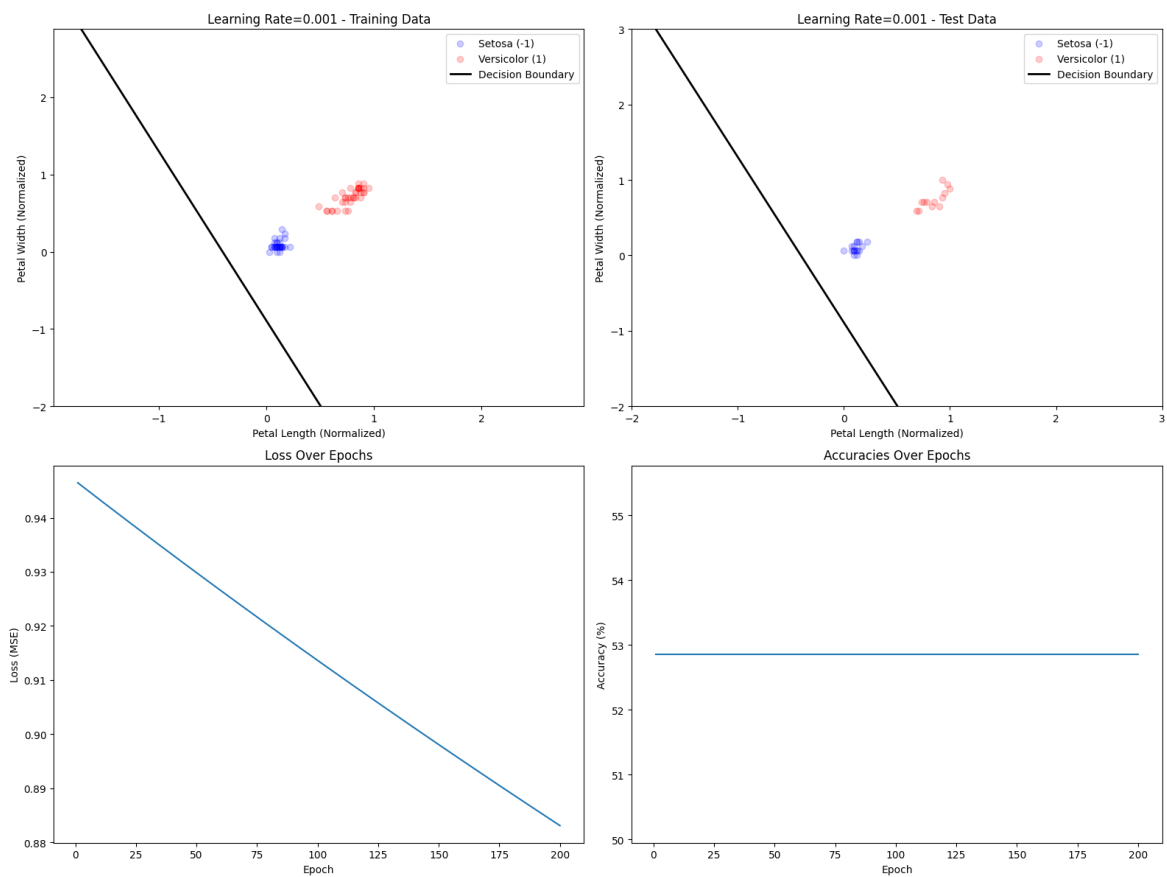
شکل ۶۳



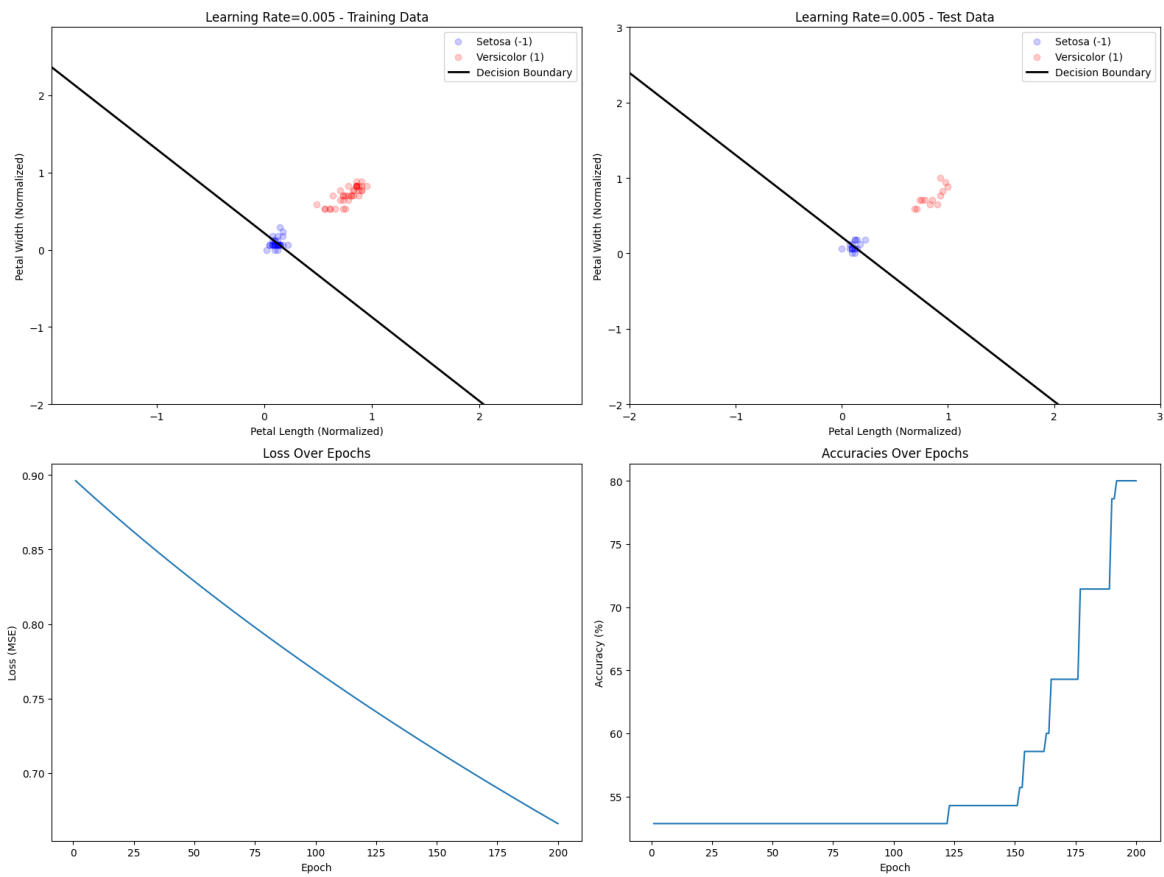
شکل ۶۴

به همین دلیل نتایج را با ۲۰۰ اپاک نیز بررسی می کنیم و به سوالات انتهایی بر اساس این نتایج

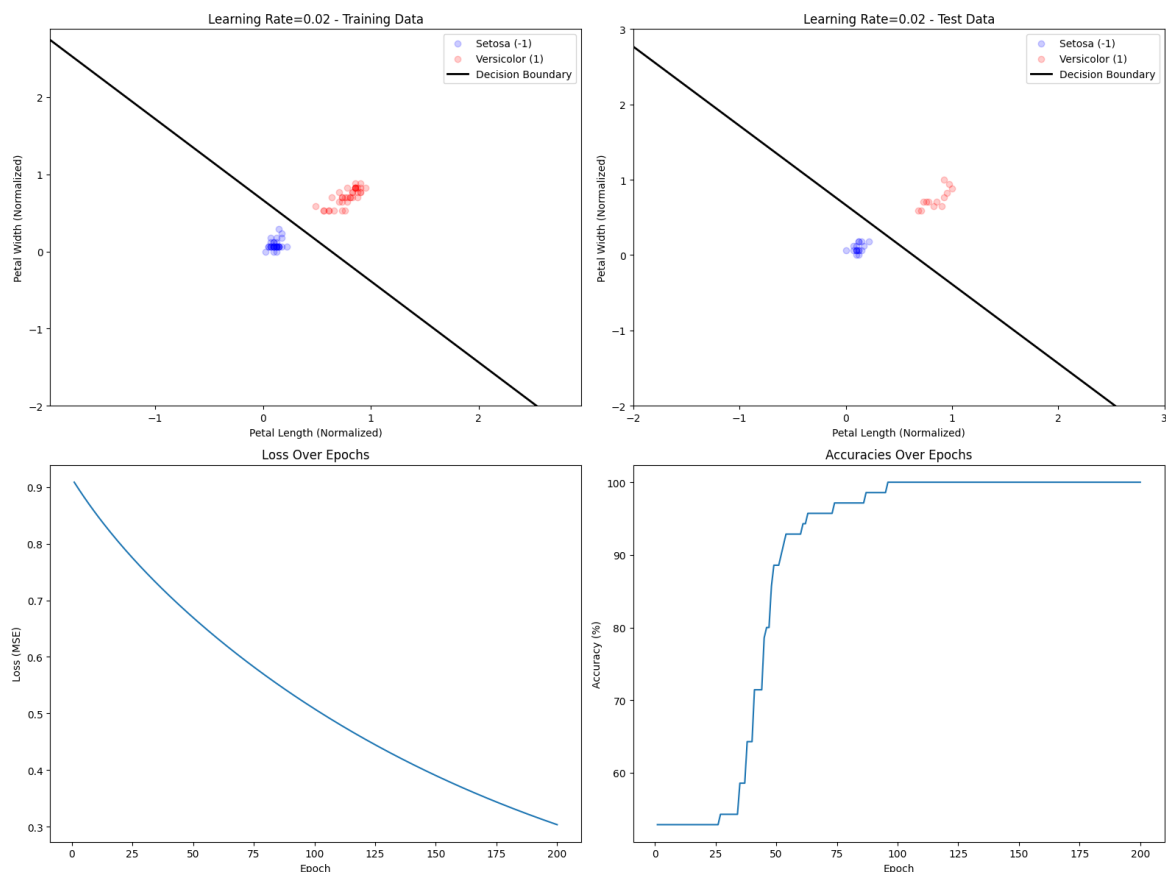
پاسخ می دهیم.



شکل ۶۵



شکل ۶۶



شکل ۶۷

تحلیل نتایج

تحلیل نتایج برای ۲۰۰ اپاک انجام شده است.

- آیا هر سه مدل همگرا شد؟ اگر خیر، چه عواملی مؤثر بودند؟
خیر؛ با اینکه هر سه مدل به سمت کاهش تابع زیان همگرا بودند، اما تنها مدل با نرخ یادگیری ۰.۰۲ به نتیجه مطلوب دست یافت.
- تأثیر جداسازی خطی داده‌ها بر عملکرد Adaline چیست؟ در صورتی که دادگان بصورت خطی جدا پذیر نباشد عملکرد این الگوریتم چگونه خواهد بود؟
اگر داده‌ها خطی جدایی پذیر باشند، الگوریتم Adaline می‌تواند یک مرز تصمیم خطی مناسب بیابد و به دقت بالایی دست یابد. در این حالت، مدل با مقدار مناسبی از اپاک و نرخ یادگیری همگرا شده و خطای آن کاهش می‌یابد.

اما اگر داده‌ها خطی جدایی‌پذیر نباشند، Adaline قادر به یافتن یک مرز تصمیم مؤثر نخواهد بود، زیرا این مدل فقط می‌تواند روابط خطی را یاد بگیرد. در این شرایط، مدل حتی با افزایش ایپاک‌ها یا تنظیم نرخ یادگیری نیز به عملکرد مطلوبی نمی‌رسد با این حال سعی می‌کند که مجذور مربعات خطا را کاهش دهد. برای حل این مشکل، می‌توان از مدل‌های غیرخطی مانند MLP یا اضافه کردن ویژگی‌های جدید و تبدیل فضای داده‌ها استفاده کرد.

- تأثیر نرخ یادگیری در عملکرد مدل را تحلیل کنید.

نرخ یادگیری کم: مدل به آرامی یاد می‌گیرد و ممکن است برای رسیدن به مقدار بهینه به تعداد ایپاک‌های بسیار زیادی نیاز داشته باشد. در مواردی، اگر مقدار آن بیش از حد کوچک باشد، مدل ممکن است قبل از یادگیری کامل متوقف شود.

نرخ یادگیری زیاد: مدل سریع‌تر تغییر می‌کند، اما این ممکن است باعث نوسانات زیاد در مقدار خطا شود و مدل از مقدار بهینه عبور کرده و همگرا نشود. در موارد شدید، مدل ممکن است به کلی واگرا شده و نتواند هیچ الگویی را یاد بگیرد.

در این مثال با افزایش نرخ یادگیری، مدل سریعتر همگرا شد.

پرسش ۴. آموزش اتوانکودر و طبقه بندی با دیتاست MNIST

مقدمه

در این بخش توضیحاتی درباره سوال داده شده بود و چیزی برای پاسخگویی وجود نداشت.

دانلود و پیش پردازش داده ها

متأسفانه به علت تحریم امکان دانلود دیتاست به کمک دستور `tf.keras.datasets.mnist` TensorFlow وجود نداشت به همین جهت فایل جداگانه دانلود (با استفاده از فیلتر شکن روی مرورگر) و در کنار کد قرار گرفت، سپس به کمک قطعه کد زیر (شکل ۴.۱) در کد `load` شده و سپس به کمک قطعه کد های بعدی (شکل ۴.۲) به صورت برداری در آمده و نرمال شدند.

```

Loading the dataset

import numpy as np
import tensorflow as tf

with np.load("mnist.npz") as data:
    x_train, y_train = data["x_train"], data["y_train"]
    x_test, y_test = data["x_test"], data["y_test"]

print(f"x_train.shape = {x_train.shape}")
print(f"x_test.shape = {x_test.shape}")

x_train.shape = (60000, 28, 28)
x_test.shape = (10000, 28, 28)
```

شکل ۶۸

```

Flattening the data since each data is a 28*28 matrix, in order to use it in our network we shall flatten it to a
784-dimensional vector

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

print(f"x_train.shape = {x_train.shape}")
print(f"x_test.shape = {x_test.shape}")

x_train.shape = (60000, 784)
x_test.shape = (10000, 784)

Normalising the data in order to avoid bias in our network due to the size of the input, we need to normalise
the data in the [0, 1] range

x_train = x_train.astype("float64") / 255.0
x_test = x_test.astype("float64") / 255.0

x_train[32].max()
np.float64(0.996078431372549)
```

شکل ۶۹

طراحی و پیاده سازی مدل

پیاده سازی و آموزش اتوانکودر ها:

در این بخش شبکه های اتوانکودر با مشخصات داده شده (لایه پنهانی با ۱۲۸ نورون و تابع ReLU و لایه خروجی _ فضای کاهش یافته _ ۴ و ۸ بعدی) تعریف شدند. با توجه به اینکه لایه میانی هر دو شبکه یکی بود و تنها بعد فضای کاهش یافته آنها متفاوت بود برای جلوگیری از تکرار کد، تابعی جهت تولید این شبکه نوشته شد و سپس دو مدل با اپتیمایزر adam و تابع mse به عنوان تابع هزینه برای مدت ۴۰ epoch ترین شدند (شکل ۴.۳)

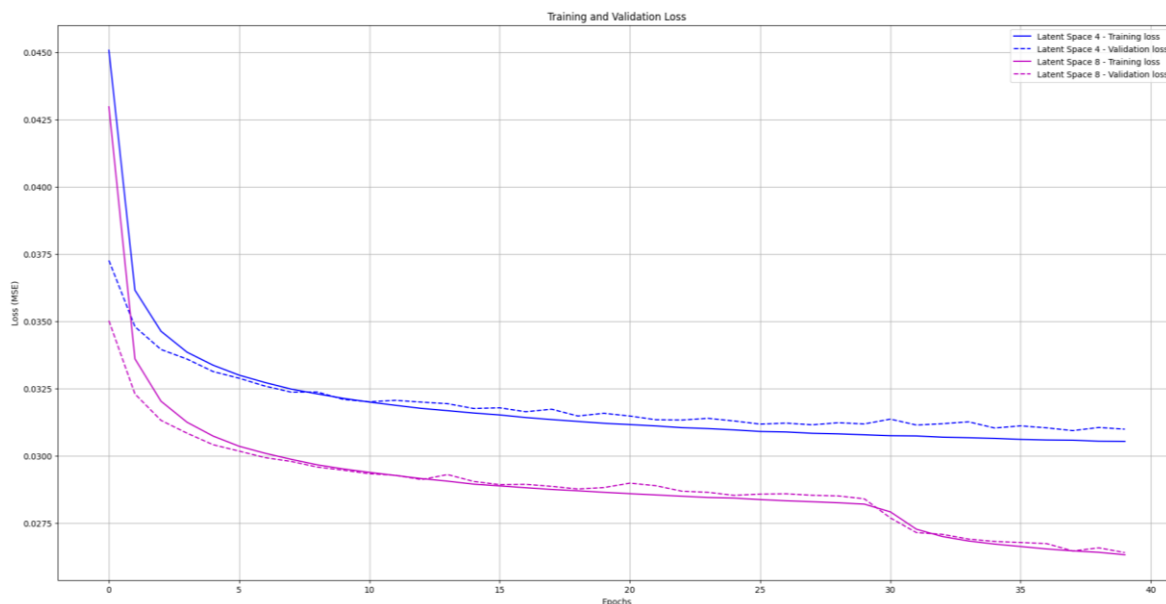
```
1 from tensorflow.keras.models import Model
2 from tensorflow.keras.layers import Input, Dense
3
4 def build_autoencoder(latent_space_dimension):
5     #Encoder
6     encoder_input = Input(shape=(784,))
7     middle_layer = Dense(
8         128,
9         activation="relu"
10    )(encoder_input)
11
12    latent_output = Dense(
13        latent_space_dimension,
14        activation="relu",
15        name="latent_layer"
16    )(middle_layer)
17
18    encoder = Model(
19        encoder_input,
20        latent_output,
21        name="Encoder"
22    )
23
24    #Decoder
25    decoder_input = Input(
26        shape=(latent_space_dimension,)
27    )
28
29    middle_layer = Dense(
30        128,
31        activation="relu"
32    )(decoder_input)
33
34    decoder_output = Dense(
35        784,
36        activation="linear"
37    )(middle_layer)
38
39    decoder = Model(
40        decoder_input,
41        decoder_output,
42        name="Decoder"
43    )
44
45    autoencoder_input = encoder_input
46    autoencoder_output = decoder(latent_output)
47    autoencoder = Model(autoencoder_input, autoencoder_output, name="Autoencoder")
48
49    return encoder, decoder, autoencoder
50
```

```
1 (encoder_latent_space_8,
2  decoder_latent_space_8,
3  autoencoder_latent_space_8) = build_autoencoder(
4      latent_space_dimension=8
5  )
6 (encoder_latent_space_4,
7  decoder_latent_space_4,
8  autoencoder_latent_space_4) = build_autoencoder(
9      latent_space_dimension=4
10 )
11
12 autoencoder_latent_space_8.compile(
13     optimizer="adam",
14     loss="mse"
15 )
16 autoencoder_latent_space_4.compile(
17     optimizer="adam",
18     loss="mse"
19 )
20
21 autoencoder_latent_space_8_history = autoencoder_latent_space_8.fit(
22     x_train,
23     x_train,
24     epochs=40,
25     batch_size=32,
26     validation_data=(x_test, x_test)
27 )
28 autoencoder_latent_space_4_history = autoencoder_latent_space_4.fit(
29     x_train,
30     x_train,
31     epochs=40,
32     batch_size=32,
33     validation_data=(x_test, x_test)
34 )
```

شکل ۷۰

بعد از ترین مدل های، نمودار تابع loss آنها برای داده های train و validation به کمک شکل ۴.۴ رسم شده و حاصل به صورت شکل ۴.۵ در آمدند که نشان میدهد هر دو شبکه بدون overfit شدن ترین شدند. ولی همانطور که از شکل ۴.۵ مشخص است شبکه ای که فضای کاهش یافته آن ۴ بعدی بود بد تر از شبکه ۸ بعدی عمل میکند که میتواند به این علت باشد که فضای اعداد ورودی به بیشتر از ۴ بعد نیاز

دارد تا بدون از دست دادن اطلاعات بتوان آنها را نشان داد. همچنین میتوان برای شبکه ۸ بعدی دید که شاید اگر برای مدت بیشتری آموزش داده میشد، نتیجه بهتری گرفته میشد ولی متأسفانه به علت محدودیت های سخت افزاری امکان این کار فراهم نبود



شکل ۷۱

طبقه بندی با انکودر:

بعد از آموزش Auto encoder ها با توجه به مشخصات داده شده قسمت encoder آنها جدا شدند و جهت جلوگیری از به روز شدن وزن آن ها freeze شدند و شبکه های feed forward جلوی آنها تعریف شدند. همچنین چون چندین کلاس خروجی داشتیم، خروجی ها به صورت one-hot در آمدند.

علاوه بر دو مدل گفته شده در صورت سوال به جهت اینکه شبکه feed forward داخل صورت سوال برای ۸ بعدی ۳ لایه و برای ۴ بعدی ۲ لایه دارد حدس زدیم که شاید اشتباهی رخ داده و یک شبکه با

ورودی ۴ بعدی و ۳ لایه هم تعریف کردیم که در نهایت performance بهتری نسبت به شبکه ۴ بعدی سوال داشت.

این شبکه ها به کمک کد های شکل ۴.۶ تعریف و آموزش داده شدند و سپس به کمک کد های شکل ۴.۷ نمودار های مربوطه برای loss و accuracy رسم شدند و نتیجه آنها در شکل ۴.۸ قابل مشاهده است.

```
1 from tensorflow.keras.models import Model
2 from tensorflow.keras.utils import to_categorical
3 from tensorflow.keras.layers import Dense, Input
4
5 y_train_onehot = to_categorical(
6     y_train,
7     num_classes=10
8 )
9 y_test_onehot = to_categorical(
10    y_test,
11    num_classes=10
12 )
13
14 def build_classifier(
15     encoder,
16     latent_dimension,
17     extra_layer=True
18 ):
19     encoder.trainable = False
20     input_layer = Input(shape=(784,))
21     latent_output = encoder(input_layer)
22
23     first_hidden_layer = Dense(
24         latent_dimension,
25         activation="relu"
26     )(latent_output)
27
28     middle_layer = Dense(
29         4,
30         activation="relu"
31     )(first_hidden_layer)
32
33     if extra_layer:
34         middle_layer = Dense(
35             4,
36             activation="relu"
37         )(middle_layer)
38
39     output_layer = Dense(
40         10,
41         activation="softmax"
42     )(middle_layer)
43
44     classifier = Model(
45         input_layer,
46         output_layer
47     )
48     classifier.compile(
49         loss="categorical_crossentropy",
50         optimizer="adam",
51         metrics=["accuracy"]
52     )
53
54     return classifier
55
56
57
```

```
1 classifier_8 = build_classifier(
2     encoder_latent_space_8,
3     latent_dimension=8
4 )
5
6
7 classifier_4_extra_layer = build_classifier(
8     encoder_latent_space_4,
9     latent_dimension=4
10 )
11 classifier_4 = build_classifier(
12     encoder_latent_space_4,
13     latent_dimension=4,
14     extra_layer=False
15 )
16
17 classifier_8_history = classifier_8.fit(
18     x_train,
19     y_train_onehot,
20     epochs=40,
21     batch_size=32,
22     validation_data=(x_test, y_test_onehot)
23 )
24 classifier_4_history = classifier_4.fit(
25     x_train,
26     y_train_onehot,
27     epochs=40,
28     batch_size=32,
29     validation_data=(x_test, y_test_onehot)
30 )
31 classifier_4_extra_layer_history = classifier_4_extra_layer.fit(
32     x_train,
33     y_train_onehot,
34     epochs=40,
35     batch_size=32,
36     validation_data=(x_test, y_test_onehot)
37 )
```

شکل ۷۲

```

1 def plot_classifier_history():
2     colors = ['b', 'g', 'r', 'k', 'm']
3     def plot_training_history(history, label, name, color_number):
4         plt.plot(
5             history.history[name],
6             label=f'{label} - Training {name}',
7             linestyle="-",
8             color=colors[color_number]
9         )
10        plt.plot(
11            history.history[f'val_{name}'],
12            label=f'{label} - Validation {name}',
13            linestyle="--",
14            color=colors[color_number]
15        )
16
17    plt.figure(figsize=(24, 12))
18    plt.subplot(1, 2, 1)
19    plt.title("Training and Validation Loss")
20    plt.grid()
21    plt.xlabel("Epochs")
22    plt.ylabel("Loss (MSE)")
23
24
25
26    plot_training_history(
27        history=classifier_8_history,
28        label="classifier with latent space 8",
29        name="loss",
30        color_number=1
31    )
32    plot_training_history(
33        history=classifier_4_history,
34        label="classifier with latent space 4",
35        name="loss",
36        color_number=2
37    )
38    plot_training_history(
39        history=classifier_4_extra_layer_history,
40        label="classifier with latent space 4 + extra layer",

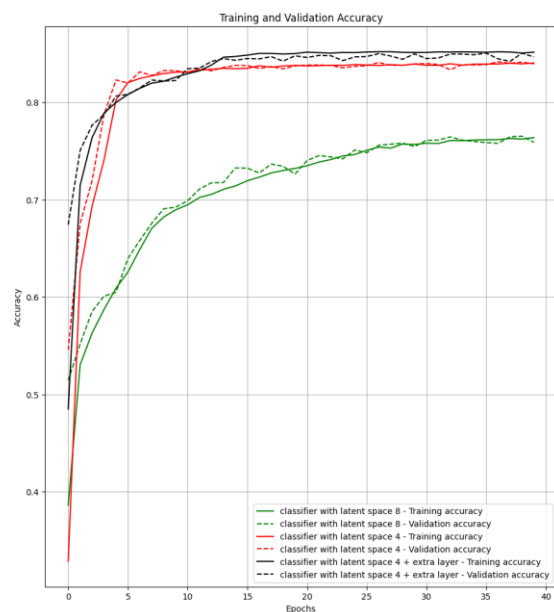
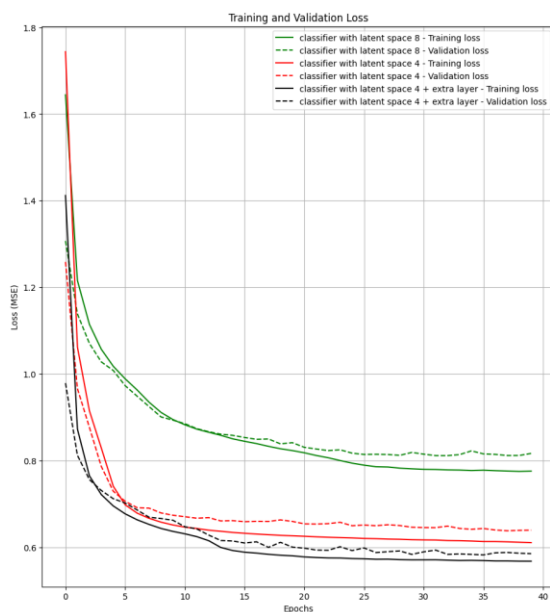
```

```

44    plt.legend()
45
46    plt.subplot(1, 2, 2)
47    plt.title("Training and Validation Accuracy")
48    plt.grid()
49    plt.xlabel("Epochs")
50    plt.ylabel("Accuracy")
51
52    plot_training_history(
53        history=classifier_8_history,
54        label="classifier with latent space 8",
55        name="accuracy",
56        color_number=1
57    )
58    plot_training_history(
59        history=classifier_4_history,
60        label="classifier with latent space 4",
61        name="accuracy",
62        color_number=2
63    )
64    plot_training_history(
65        history=classifier_4_extra_layer_history,
66        label="classifier with latent space 4 + extra layer",
67        name="accuracy",
68        color_number=3
69    )
70
71    plt.legend()
72    plt.show()
73
74 plot_auto_encoder_history()
75 plot_classifier_history()

```

شکل ۷۳



شکل ۷۴

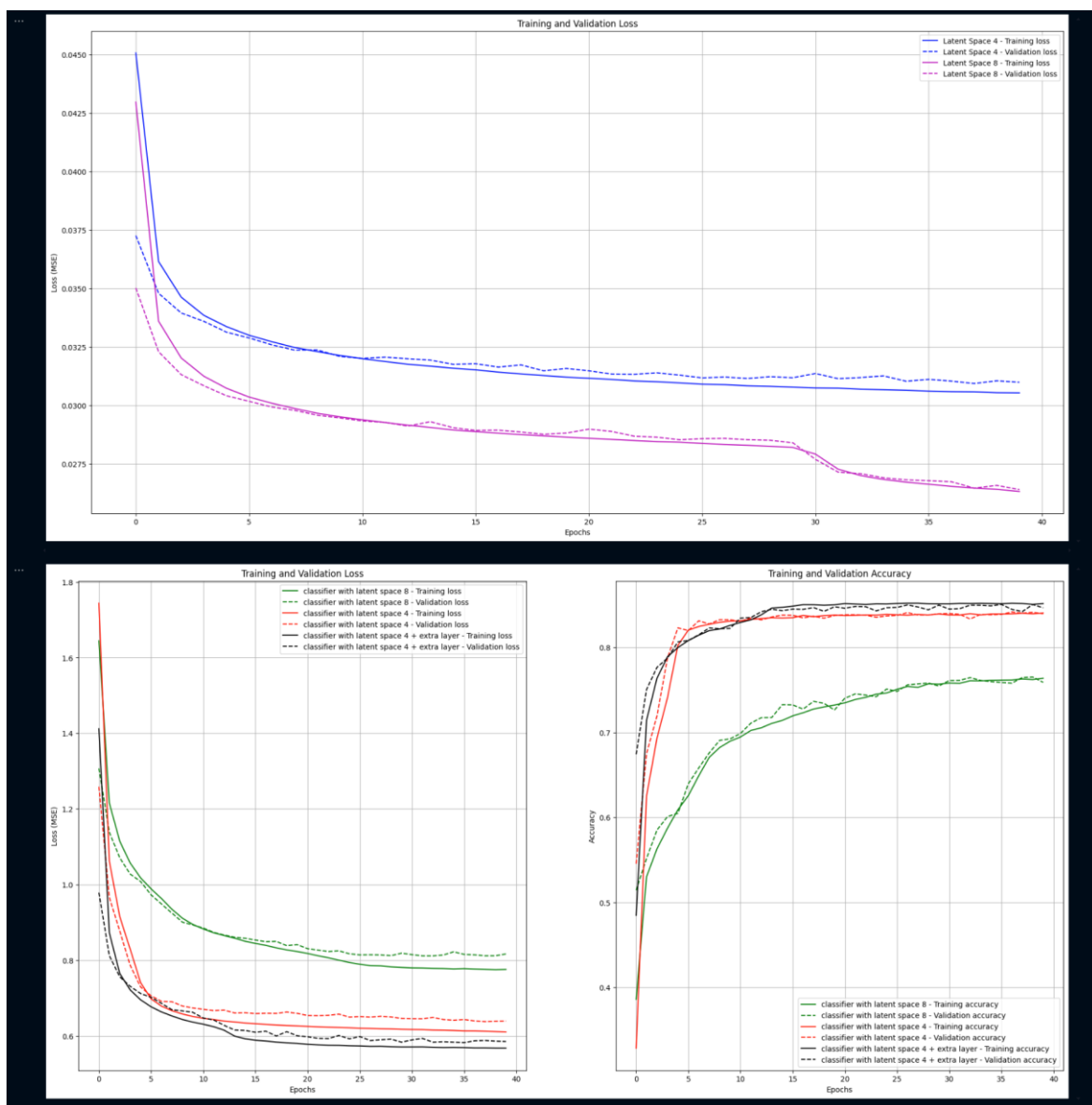
همانطور که از نمودار های شکل ۴.۸ مشخص است، شبکه ۴ بعدی که مطابق صورت سوال تعریف شده بود بدترین نتیجه را داشت و بعد از آن شبکه ۸ بعدی و بهترین نتیجه هم برای شبکه ۴ بعدی با لایه های feed forward اضافه بود. این نشان میدهد که گرچه با توجه به شکل ۴.۴ شبکه انکودر ۴ بعدی اطلاعاتی را از دست داده بود ولی با اضافه کردن لایه های طبقه بندی بیشتر توانسته بود عملکرد بهتری داشته باشد.

همچنین این نشان میدهد که شبکه طبقه بندی ۸ بعدی بهترین عملکرد ممکن را نداشته است و احتمالاً با تغییر ساختمان شبکه میتوان نتیجه بهتری گرفت!

نتایج و تحلیل

نتایج:

نمودار های مربوط به شبکه ها به همراه توضیحاتشان در بالا آمده است ولی در اینجا هم مجدداً در شکل ۴.۹ آورده شده اند (در بخش تحلیل هم تحلیل نمودار ها با جزییات بیشتری آمده است) تعداد پارامتر های مدل ها هم به شرح رو به رو هستند: مدل هایی که در صورت سوال ذکر شده بودند به همان صورت و با تابع categorical cross entropy و بهینه ساز adams تعریف شدند. شبکه اضافی هم (برای بخش امتیازی) از همان اتوانکودر ۴ بعدی استفاده میکرد ولی در بخش طبقه بندی یک لایه اضافی با ۴ نورون برایش تعریف شده بود.



شکل ۷۵

تحلیل و مقایسه

مقایسه خطای بازسازی: با توجه به شکل ۴.۹ مدل با ۸ نورون باز سازی بهتری داشته است، این امر میتواند از این ناشی باشد که ۴ نورون برای نمایش فضای ورودی ها کافی نیست (تعداد فضا های پوچی به اندازه کافی نیستند) البته میتواند ناشی از این هم باشد که برای کاهش فضا و شکستن همبستگی ها به ۴ بعد نیاز به لایه های مخفی بیشتری هست

مقایسه دقت طبقه بندی: همانطور که از نمودار های شکل ۴.۸ دیده میشود شبکه با فضای نهان ۸ (قرمز) عملکرد بهتری از شبکه با فضای نهان ۴ (سبز دارد). با توجه به اینکه این دو شبکه قسمت feed forward مشابهی داشتند میتوان نتیجه گرفت این تفاوت ناشی از ازدست رفتن اطلاعات در شبکه ۴ بعدی است.

ولی شبکه با فضای نهان ۴ بعدی که لایه اضافه در بخش classifier داشت (سیاه رنگ) عملکرد بهتری از هر دو شبکه دیگر داشت. این امر نشان میدهد هرچند که در شبکه انکودر ۴ بعدی اطلاعات بیشتری از دست رفته اند ولی قسمت طبقه بندی حداکثر استفاده را از این اطلاعات نمیکرده و با بهبود این بخش هر دو شبکه میتوانند عملکرد بهتری داشته باشند. پس میتوان نتیجه گرفت هرچند کاهش ابعاد در این بخش تاثیر گذار بوده ولی پارامتر های دیگر چون معماری بخش طبقه بندی در جواب نهایی تاثیر گذار تر هستند.

مقایسه تعداد پارامتر ها: همانطور که از شکل ۴.۴ مشخص است اگر تعداد epoch ها کمتر میبود شبکه بازدهی کمتری میداشت پس پارامتر هایی چون epoch و batch ساز تا زمانی که منجر به overfit نشوند تاثیر مثبتی دارند.

همچنین نمودار شکل ۴.۸ (نمودار سیاه رنگ) نشان میدهد که اگر تعداد لایه های بخش طبقه بندی بیشتر میبود (تا زمانی که overfit رخ نداده) دقت شبکه بالاتر میرفت!

نشانه های overfit: در شبکه هایی که آموزش داده شدند نشانه ای از overfit مشاهده نشد و نمودار های test و validation کاملاً هم گام پیش میرفتند ولی اگر با مشکل overfit رو به رو میشدیم، احتمالاً میشد با استفاده از dropout یا regularization (مانند سوال اول) با آنها مقابله کرد.

مدل جدید: مدل جدید تر (با رنگ سیاه در نمودار ۴.۹) در بالا تر توضیح داده شد که از همان انکودر ۴ بعدی استفاده میکند ولی بخش طبقه بندی آن به کمک لایه های اضافه به مراتب از حالت عادی انکودر ۴ بعدی (سبز رنگ) بهتر عمل میکند. احتمالاً با اعمال تغییرات در بخش اتوانکودر هم میشد حتی عملکرد بهتری از شبکه بدست آورد.

